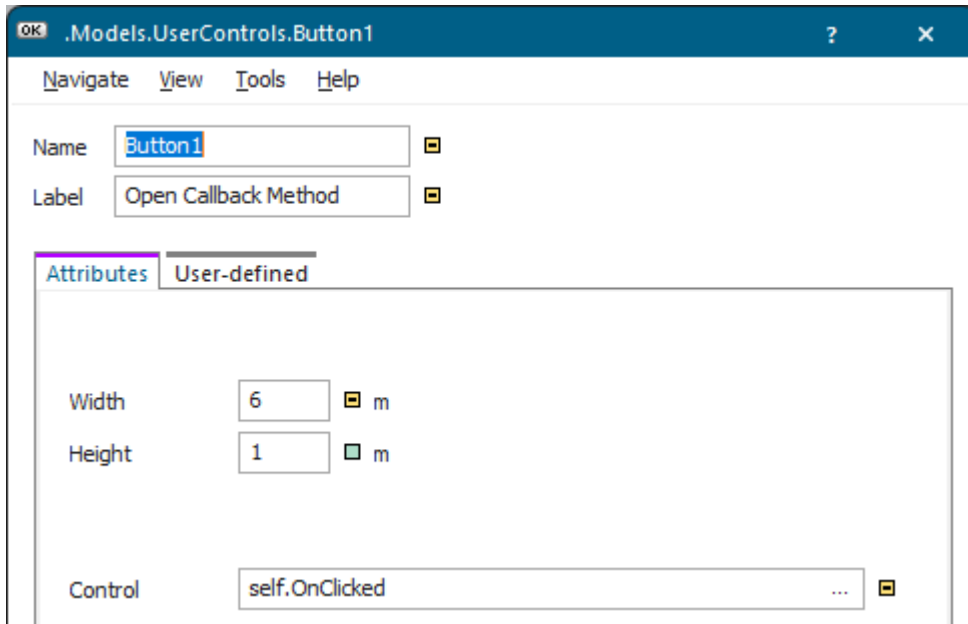
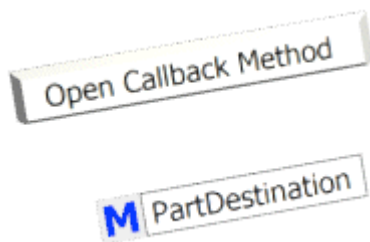




We then entered the following source code into the control that opened. It opens the *callback method* of the *sensor control* in the *Frame* with the name *PartDestination* and the label *Callback*.

As we want to open the window of the *Method*, not to execute its source code, we have to enter the reference operator **&** in front of the name of the *sensor control*.

```
self.~.~.&PartDestination.openDialog
```



Compare [Program the Control for the Action to be Executed](#)

Back to [Select an Option from a Drop-down List](#)

Importing Data for the Simulation

Import Data for the Simulation

Introduces you on how to work with data provided by the departments of the plant you are modeling, such as production lists, data, which machines output, etc. while you create your simulation model.

As a rule you will import this data from a spreadsheet program, such as Microsoft Excel, or from a database, into a *Plant Simulation* list or table. From the data bulk you will have to filter out the data you actually do need to use for your simulation project. The material flow objects can then access these lists and use the data for the simulation.

Naturally you may also want to export some or all of the data, which the simulation runs created, so that you can use and manipulate it in other programs.

Importing and Exporting Data

Import and Export Data

Plant Simulation provides a number of ways to import data from other applications into *Plant Simulation*. In some cases you will import data into an *Plant Simulation* list or a table, process it there and then pass it on to the material flow objects. In other cases you will directly provide imported data to the material flow objects, for example by using the *FileInterface*.

You can:

- [Import a Text File or an Object File into a List](#)
- [Import Data from a Microsoft Excel Worksheet](#)
- [Import a List of Services, Shifts, etc. into an Object](#)
- [Import Data in XML Format](#)
- [Import Data from a Database](#)
- [Import or Export Data in ANSI Format](#)
- [Import or Export the Contents of a List](#)

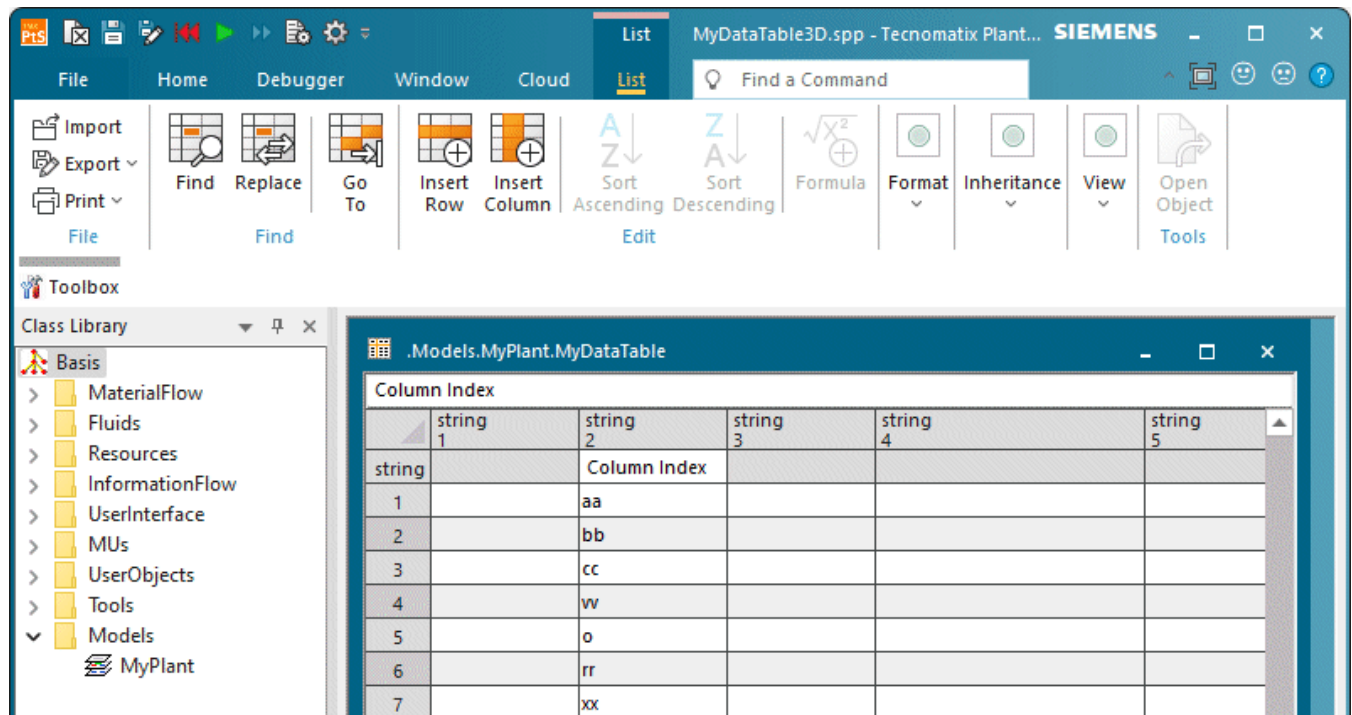
Compare [Database Interfaces](#)

[Back to Import Data for the Simulation](#)

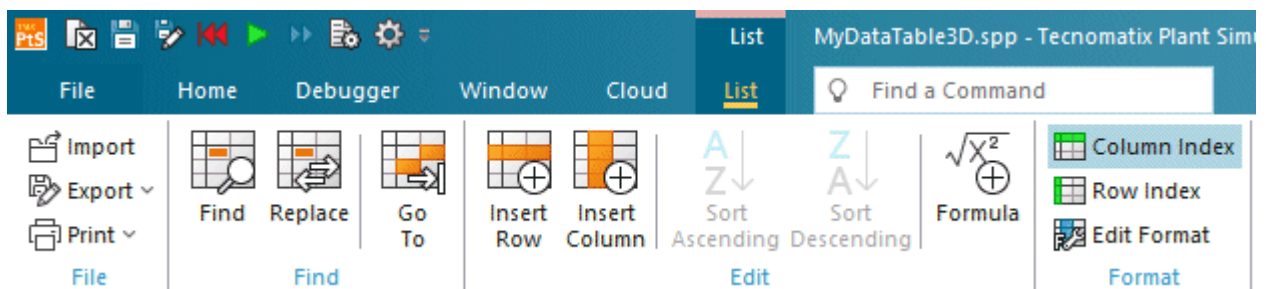
Import a Text File or an Object File into a List

The *Plant Simulation* lists [DataTable](#) , [DataList](#) , [DataQueue](#) , [DataStack](#) , and [TimeSequence](#)  can open text files, *Plant Simulation* object files, XML files, and Microsoft Excel files, which contain the data you want to use in your simulation project.

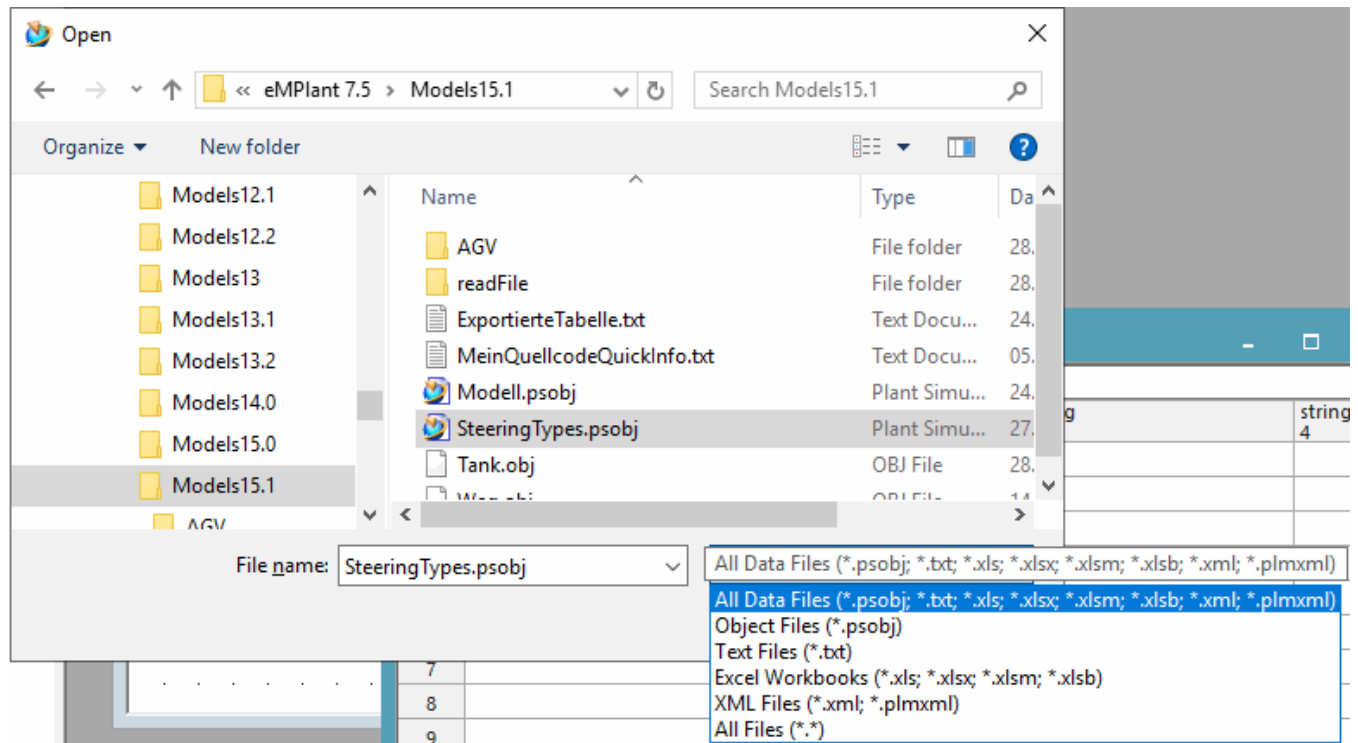
- Before you open a text file, we advise to select the correct data type that fits the data you are going to import into the individual columns of the *DataTable*. If the text file has a column header, which you want to also use in your *Plant Simulation* list, activate and show the column index in the *Plant Simulation* list.



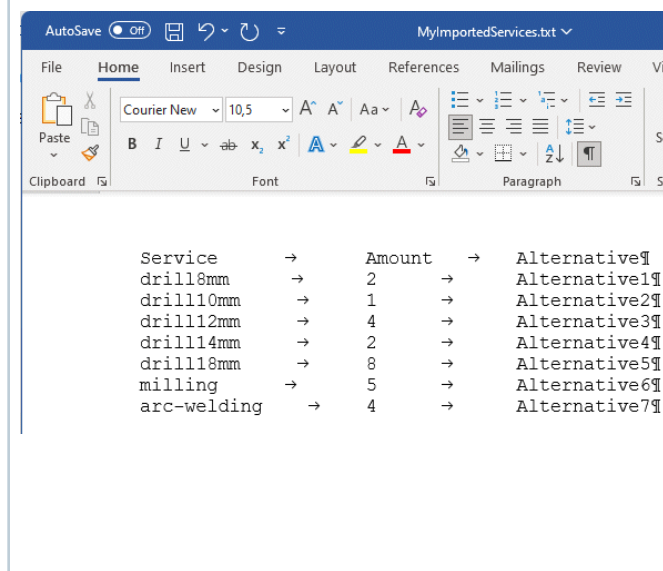
- To open a text file or an *Plant Simulation* object file in a list, click **Import File** on the **List** ribbon tab.



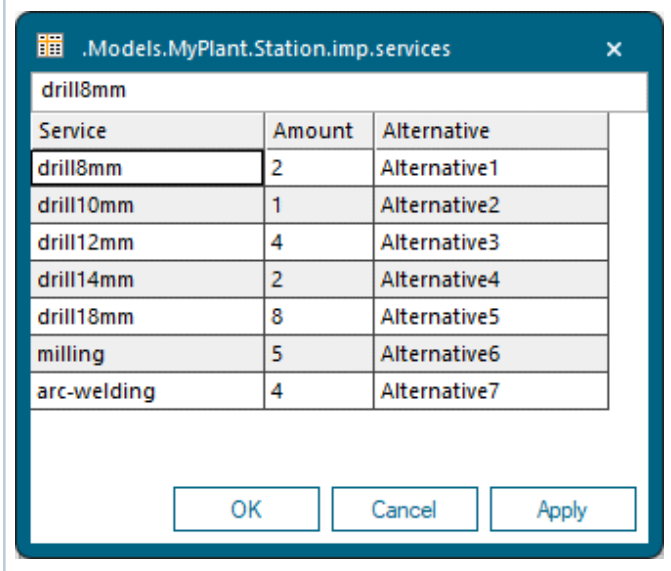
- Navigate to the folder where you stored the file. Select the type of file you would like to open and click **Open**.



Opened Text file, example



Opened Plant Simulation object file, example



Plant Simulation opened the object file with the formatting, which the original file had. For the text file we had to manually select the correct data types.

- To export the contents of an *Plant Simulation* list or table, click **Export to File** or **Export Object File** on the **List** ribbon tab.

Compare Set the Data Type of a Column

Compare Import or Export the Contents of a List

Compare Context Menu of Embedded Lists

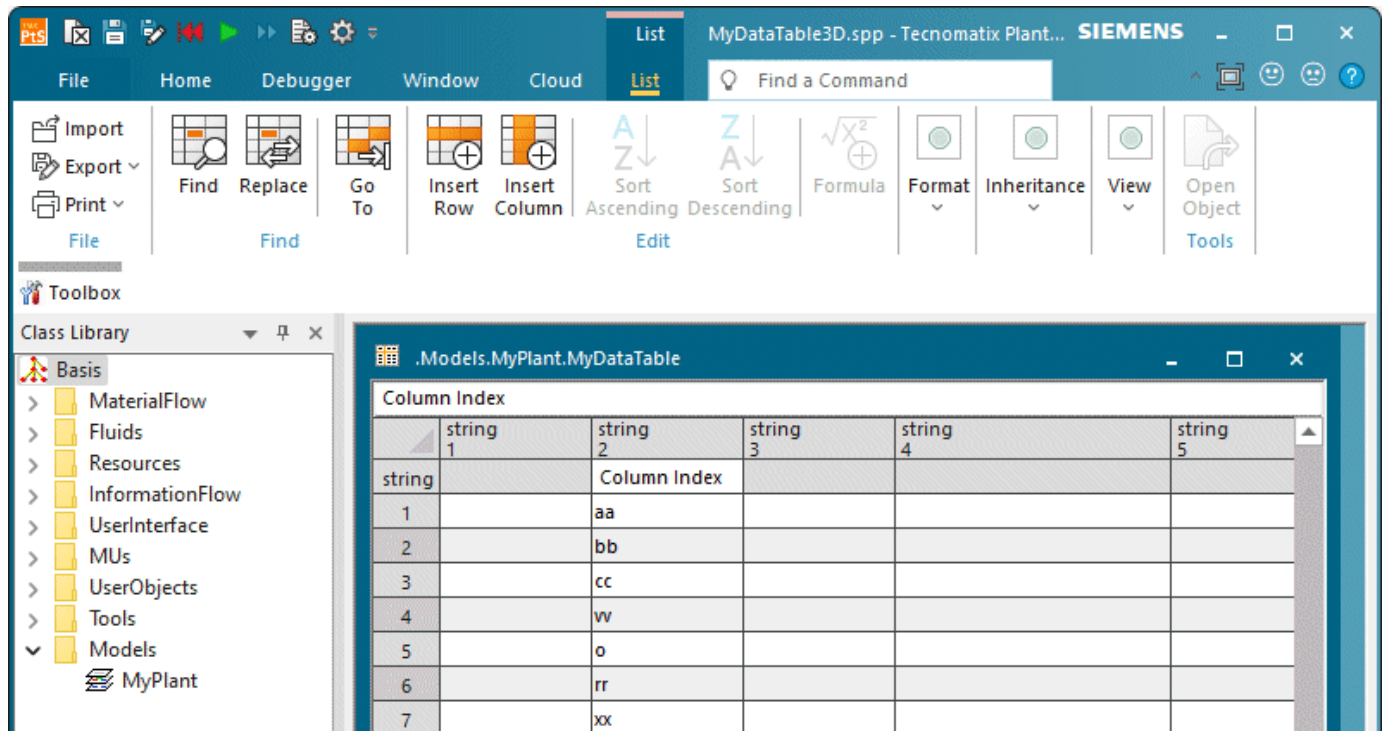
Back to Import and Export Data

Import Data from a Microsoft Excel Worksheet

When *Plant Simulation* reads an MS Excel table, it attempts to convert the values in the individual columns to the data types of the columns in your *Plant Simulation* table. This only works, when you created the columns on the Excel worksheets so that they only contain a single data type, meaning that, when you, for example, assign the data type *String* to a cell of a column, the entire column may only be of data type *String*.

After importing the data into *Plant Simulation*, you have to check the data types of the individual columns, you might have to correct the type of some, and you will have to filter out data you do not need. You can either do this manually in the open *Plant Simulation* table, or you can program a *Method* to manipulate the data while importing it.

If the Excel file has a column header, which you also want to use in your *Plant Simulation* list, activate and show the column index in the *Plant Simulation* list.



- For our example we used the file *MyTestData.xlsx*, which looks like this in Excel.

AutoSave Off MyTestDa... - Saved

File Home Insert Page Layout Formulas Data Review View Help PDF-XChange

Clipboard Font Alignment Number Styles Cells Editing Ideas Sensitivity

K19 06.02.2004 06:25:46.000

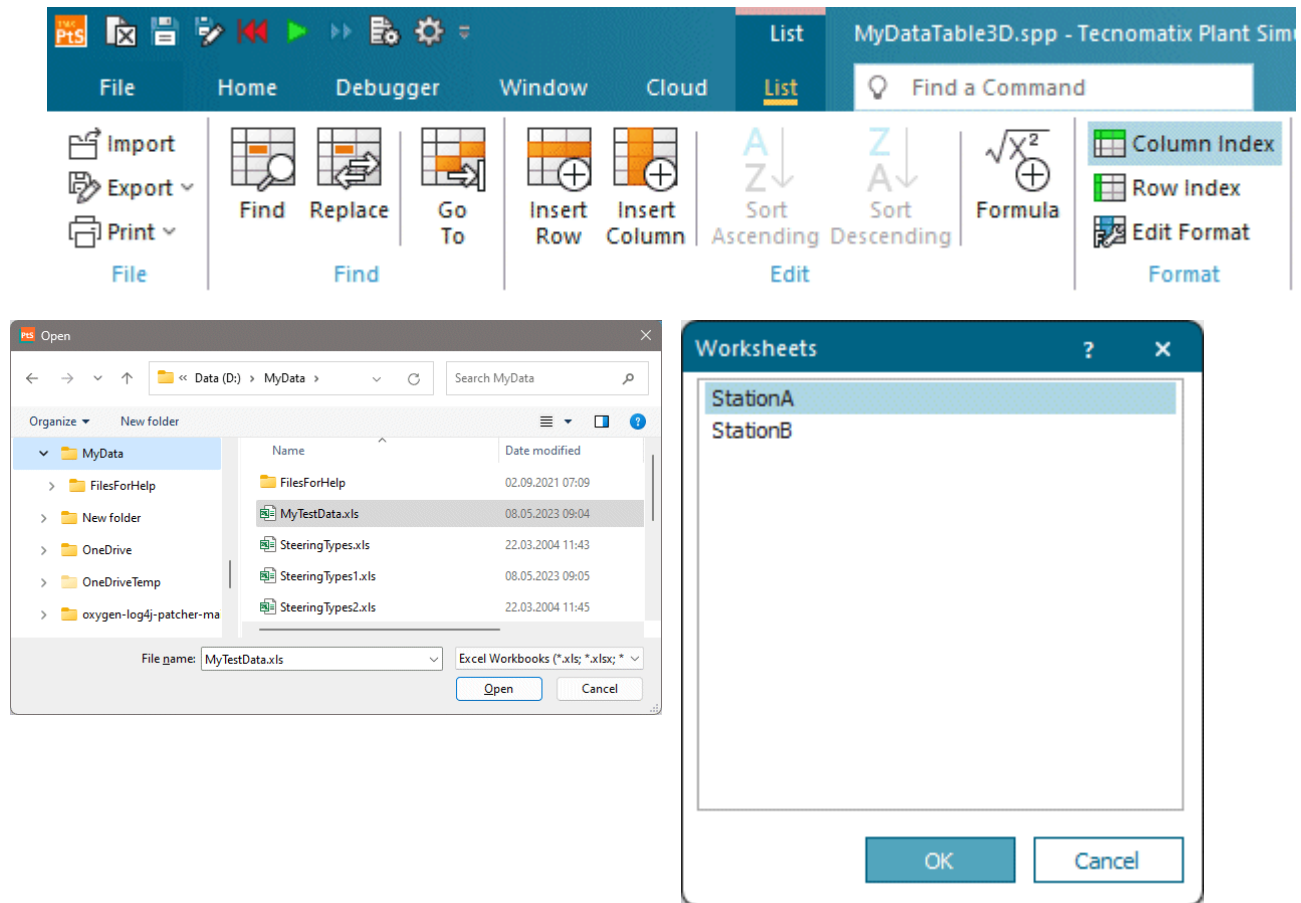
Station	Section	Line	ID	Status	Type	Codes	Actions	planned Assembly	Drawing-No.	Arrival Date	Delivery-No.
Station/Shipping: All stations, conveyors, and shipping, area 1, starting 07.02.2004 06:00:00, produced, tested, shipped, partially, in toto											
Data starting 07.02.2004 14:43:55											
Sorted by: Date and Area											
31	70	70	6750461331	MM	467500	L ,FCL ,HA ,I265,M002		01.02.2004	3	06.02.2004 06:07:42.000	0
33	70	99	6610462552	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:08:28.000	8100176
33	70	99	6610462553	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:09:15.000	8100176
31	70	70	6700837672	MM	467000	L ,FC ,HA ,I247,M002		02.02.2004	2	06.02.2004 06:10:02.000	0
31	70	70	6700837670	MM	467000	L ,FC ,HA ,I247,M002		02.02.2004	2	06.02.2004 06:11:02.000	0
31	70	99	5492016531	MM	454900	L ,FC ,HA ,I327,M112		02.02.2004	2	06.02.2004 06:11:46.000	12100861
31	70	99	5492016530	MM	454900	L ,FC ,HA ,I327,M112		02.02.2004	2	06.02.2004 06:12:35.000	12100861
31	70	99	5492016534	MM	454900	L ,FC ,HA ,I327,M112		02.02.2004	2	06.02.2004 06:13:23.000	12100853
31	70	99	5492016532	MM	454900	L ,FC ,HA ,I327,M112		02.02.2004	2	06.02.2004 06:14:10.000	12100857
31	70	99	5492016535	MM	454900	L ,FC ,HA ,I327,M112		02.02.2004	2	06.02.2004 06:14:59.000	12100853
31	70	99	5492016536	MM	454900	L ,FC ,HA ,I327,M112		02.02.2004	2	06.02.2004 06:15:52.000	12100857
31	70	99	5492016538	MM	454900	L ,FC ,HA ,I327,M112		02.02.2004	2	06.02.2004 06:16:42.000	12100857
31	70	99	5492016537	MM	454900	L ,FC ,HA ,I327,M112		02.02.2004	2	06.02.2004 06:17:46.000	12100857
33	70	99	6610462551	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:25:46.000	8100187
33	70	99	6610462554	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:26:38.000	8100189
33	70	99	6610462555	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:28:15.000	8100189
33	70	99	6610462557	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:29:00.000	8100185
31	70	70	5542016886	MM	455400	L ,FCL ,423 ,HA ,I287 ,		02.02.2004	3	06.02.2004 06:29:58.000	0
31	70	70	5542016884	MM	455400	L ,FCL ,423 ,HA ,I287 ,		02.02.2004	3	06.02.2004 06:31:21.000	0
31	70	99	5542016892	MM	455400	L ,FCL ,423 ,HA ,I287 ,		02.02.2004	3	06.02.2004 06:32:08.000	8100185
33	70	70	6810462581	MM	468100	L ,FCL ,423 ,HA ,I327 ,		02.02.2004	3	06.02.2004 06:32:56.000	0
31	70	99	5542016890	MM	455400	L ,FCL ,423 ,HA ,I287 ,		02.02.2004	3	06.02.2004 06:34:15.000	8100175
31	70	70	5542016891	MM	455400	L ,FCL ,423 ,HA ,I287 ,		02.02.2004	3	06.02.2004 06:36:02.000	0
33	70	70	6610462612	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:36:46.000	0
33	70	70	6610462622	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:37:43.000	0
33	70	99	6610462623	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:38:54.000	8100184
33	70	99	6610462624	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:39:53.000	8100181
33	70	99	6610462613	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:41:01.000	8100182
33	70	99	6610462625	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:42:02.000	8100186
33	70	70	6610462626	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:42:53.000	0
33	70	99	6610462627	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:44:16.000	8100181
33	70	70	6610462628	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:45:07.000	0
33	70	99	6610462629	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:46:30.000	8100182
33	70	70	6610462630	MM	466100	L ,FW ,423 ,498 ,949 ,		02.02.2004	3	06.02.2004 06:47:21.000	0
31	70	99	5082016557	MM	450800	L ,FS ,HA ,I265,M002		02.02.2004	2	06.02.2004 06:50:21.000	12100859

StationA StationB

Ready 100%

- To open the Excel file in an *Plant Simulation* table, click **Import File** on the **List** ribbon tab.

Navigate to the folder where you stored the file. Select the file you would like to open and click **Open**. Select the Excel worksheet you want to import and click **OK**.



- The imported Excel worksheet looks like this in our *Plant Simulation* table.

Data starting 07.02.2004 14:43:55									
	string 1	string 2	string 3	string 4	string 5	string 6	string 7	string 8	string 9
string	Station/Shipping:...								
1	Data starting 07....								
2	Sorted by: Date a...								
3	Station	Section	Line	ID	Status	Type	Codes	Actions	planned Assem...
4	31	70	70	6750461331	MM	467500	L ,FCL ,HA ,I265,M002,M...		2004/02/01
5	33	70	99	6610462552	MM	466100	L ,FW ,423 ,498 ,949 ,HA...		2004/02/02
6	33	70	99	6610462553	MM	466100	L ,FW ,423 ,498 ,949 ,HA...		2004/02/02
7	31	70	70	6700837672	MM	467000	L ,FC ,HA ,I247,M002,M...		2004/02/02
8	31	70	70	6700837670	MM	467000	L ,FC ,HA ,I247,M002,M...		2004/02/02
9	31	70	99	5492016531	MM	454900	L ,FC ,HA ,I327,M112,M...		2004/02/02
10	31	70	99	5492016530	MM	454900	L ,FC ,HA ,I327,M112,M...		2004/02/02
11	Station	Section	Line	ID	Status	Type	Codes	Actions	planned Assem...

Check the file for correct data types, data you do not need, etc. You can also pre-process or post-process the data by programming a *Method*.

- To export the contents of a *Plant Simulation* list or table as an Excel file, click **Export Excel File** on the **List** ribbon tab.

Compare **Import or Export the Contents of a List**

Back to **Import and Export Data**

Import a List of Services, Shifts, etc. into an Object

Note

In most cases importing one of the mentioned lists involves two lists.

The main list contains any expression of your choice, which serves as a placeholder for the sublists. You might, for example, enter different names for the different stations. In the example below we entered Operations for MyPart A.

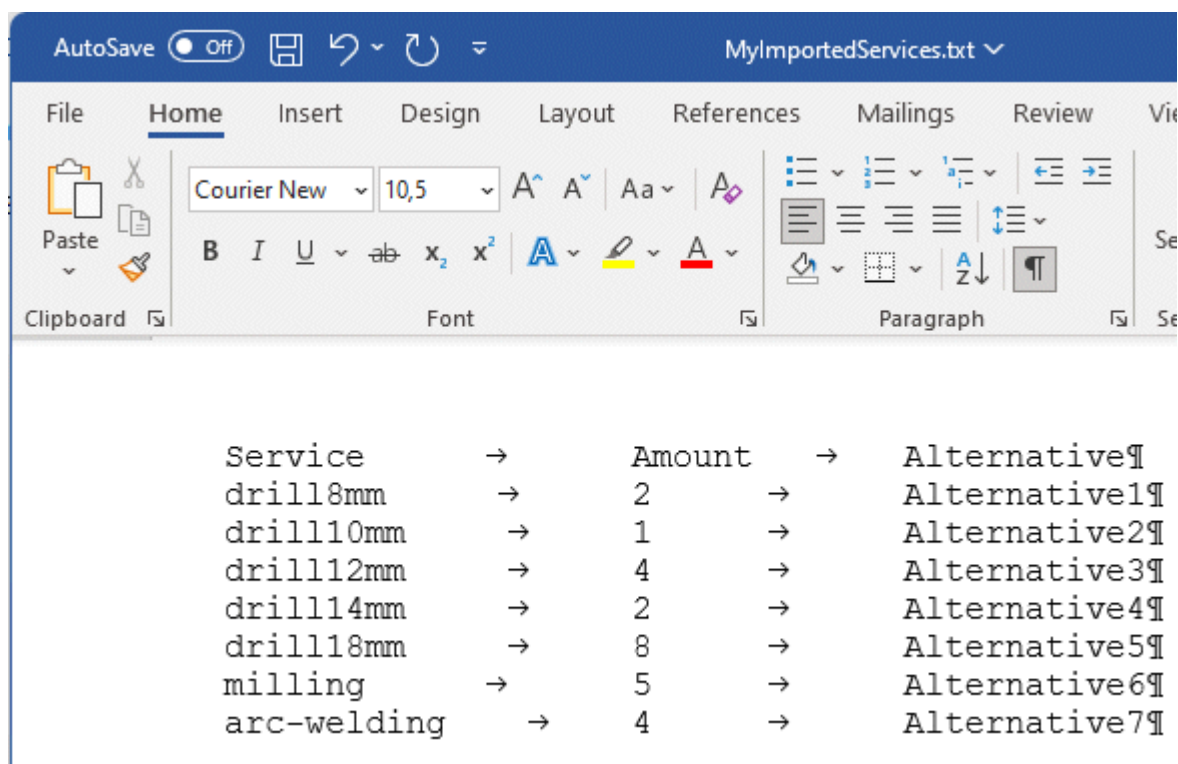
The second list, i.e., the sublist, contains the names of the actual stations and information pertaining to them.

Operations for MyPart A		
	string 0	table 1
string	Entity type	Operations
1	MyPart A	Operations for MyPart A
2	MyPart B	Operations for MyPart B
3		
4		

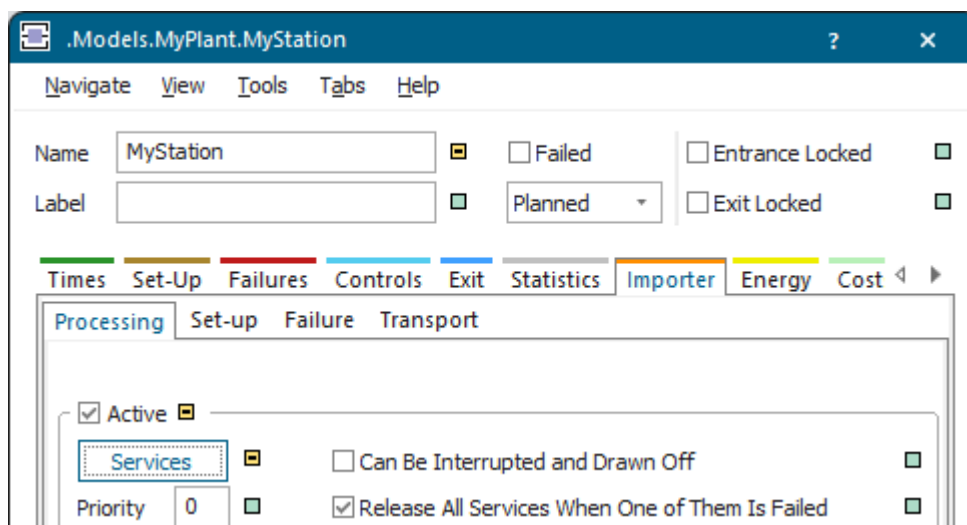
10.0000			
	object 0	time 1	time 2
string	Operation	Setup time	Processing time
1	Milling	10.0000	1:00.0000
2	Drilling_A	20.0000	2:00.0000
3	Packing	10.0000	3:00.0000
4			

To import a services list, a list of shifts or a worker creation table into one of the lists which are embedded in a *Plant Simulation* object:

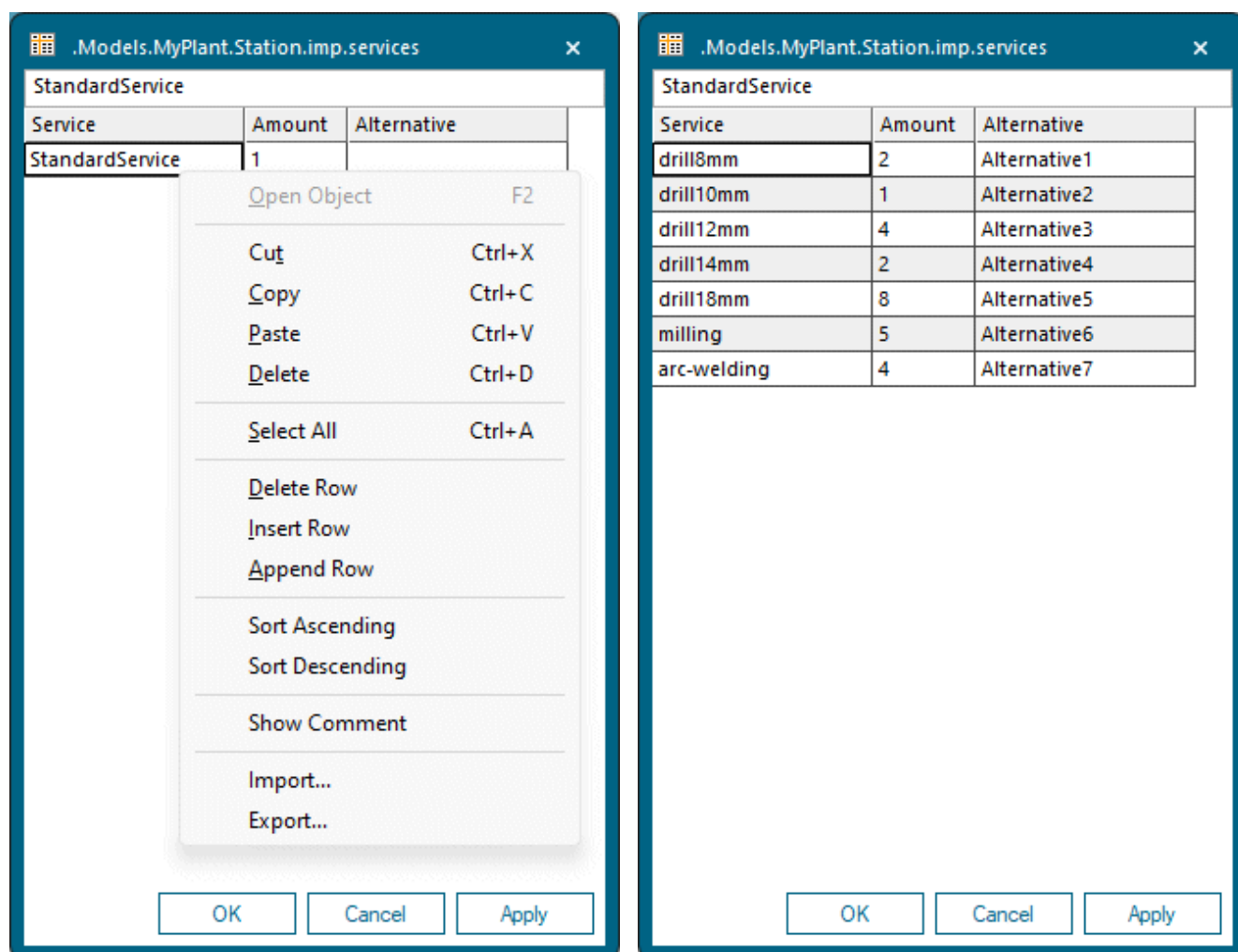
- Open the file, which you received, in a text editor. Make sure that the text, which is to be shown in the column header, is the first line in the file. Separate the columns with tabs. Save the file as a text file (.txt).



- Open the *Plant Simulation* object and change to the **tab Importer** where you want to import the file. Click the inheritance check box to the right of **Services** on the sub-tab **Processing** so that it looks like this . Click **Services**.



- Right-click in the list and click **Import**. Navigate to the folder where you saved the text file and click **OK**.



If need be, you can then manipulate the contents of the list with the commands on the **Context Menu of Embedded Lists**. You might, for example, add rows or sort the list in ascending or descending order.

- To export the contents of an embedded list as a tab-delimited text file, right-click in the list and select **Export**.

Compare Specify Times During Which the Plant Works Part of the Time

Back to [Import and Export Data](#)

Importing Data in XML Format

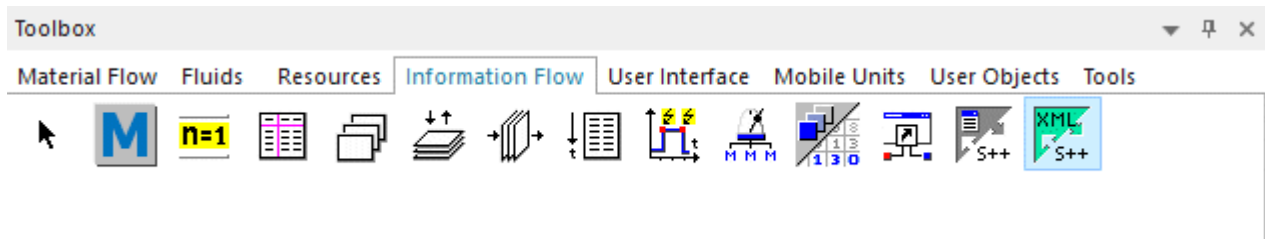
Import Data in XML Format

You can import data stored in XML format into *Plant Simulation* and extract data from the XML file. You may, for example, use the *XMLInterface* to read data, which you exported from *Process Designer*, the Siemens program for planning, analyzing and managing manufacturing processes, or from an XML database.

You can then use the data, which the *XMLInterface* imported, and which you manipulated with a *Method*, to run a simulation in *Plant Simulation*. You might afterwards use the methods `write` and `writeElement` to write the simulation results back to an XML file and continue working with these simulation results in *Process Designer* or another program.

To get the most out of the *XMLInterface*, you have to be familiar with *XPath* (*XML Path Language*) instructions. You might, for example, check the web site <http://www.w3.org/TR/xpath> for more information. In addition, you have to be familiar with programming in *SimTalk*.

You can insert the *XMLInterface*  into your simulation model from the folder **InformationFlow** in the *Class Library* or from the toolbar **Information Flow** in the *Toolbox*.



You can:

- **Select File Name, Context, and Import Method**
- **Read and Write Data Sequentially**
- **Read and Access Data Randomly**
- **Access and Traverse Data Randomly**

Back to **Import and Export Data**

Select File Name, Context, and Import Method

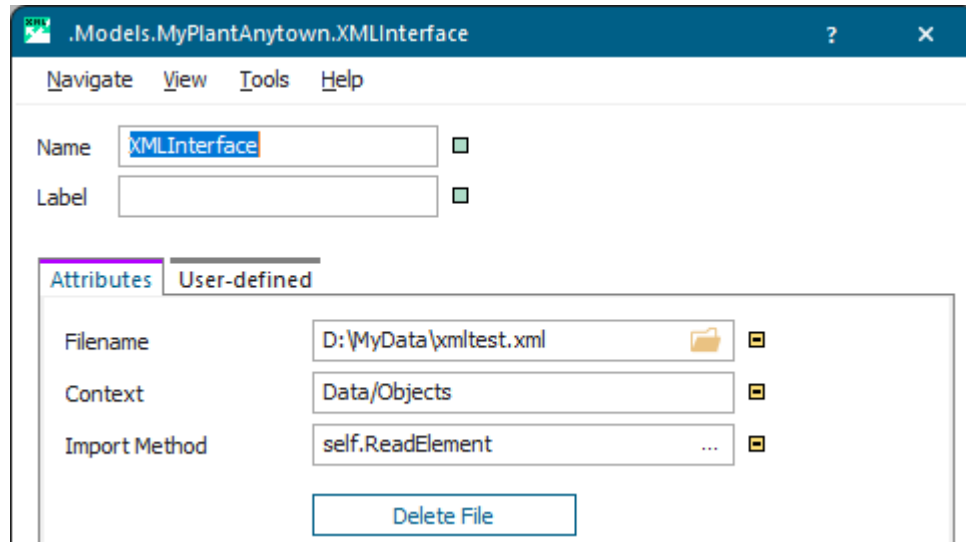
When you work with the *XMLInterface*, you will make extensive use of *XPath* (*XML Path Language*) instructions and of the *Plant Simulation* methods and attributes of the *XMLInterface* itself.

Into the dialog of the *XMLInterface* you:

- Enter the **File Name** of the XML file, which the *XMLInterface* opens, if you want to import data. When you export data, it specifies the name of the file, which it saves.
- Enter the **Context** of the data you want to import. The context designates the node of the structure of the XML document at which the *XMLInterface* starts reading data. You might, for example, enter the names of certain data, or of certain objects that you are interested in, for example `Data/Objects`. This way you can restrict the data to be read, i.e., if you do not need to work with all of the data contained in the XML file, you will enter a context.

If you do not enter a context, the *XMLInterface* imports the entire file, which may contain a large amount of data, some of which you do not need. This might take some time to import and use up a large amount of your computer's RAM.

- Click  to enter the path to and the name of the **Import Method** into the text box. In the *Method M* you program how to extract and to sequentially process the imported data.



Back to [Import Data in XML Format](#)

Read and Write Data Sequentially

You can **sequentially read the data line-by-line**, write it to an **Import Method**, and then immediately process that data line-by-line. To restrict the amount of data to be imported, enter a **Context**.

This *Method* demonstrates how to sequentially write an XML file.

The source code looks like this:

```
XMLInterface.FileName := "D:\MSXML 4.0\writeSequentially.xml"
// opens the XML document for sequential writing
XMLInterface.openWrite
XMLInterface.startElement("catalog")
XMLInterface.startElement("book")
// adds attributes to the item 'book'
XMLInterface.addAttribute("id", "bk01")
XMLInterface.addAttribute("xmlns", "myBooks")
XMLInterface.addAttribute("xmlns:aa", "specAth")
// these are the children of the item 'book'
XMLInterface.writeElement("aa:author", "Gambardella, Matthew")
// adds an attribute to the item 'author'
XMLInterface.addAttribute("age", "16")
XMLInterface.writeElement("title", "XML Developer's Guide")
XMLInterface.writeElement("genre", "Computer")
XMLInterface.writeElement("price", "44.95")
```

```
XMLInterface.writeElement("publish_date","2000-10-01")
XMLInterface.writeElement("description","An in-depth ...")
// terminates the item 'book'
XMLInterface.endElement
// terminates the element 'catalog'
XMLInterface.endElement
XMLInterface.close
```

Back to [Import Data in XML Format](#)

Read and Access Data Randomly

Plant Simulation can **read the data** in its entirety, and then **randomly** process it as a whole.

When doing this, *Plant Simulation* imports the entire document first and then processes and analyzes all data in the *Method*, which you program. Randomly accessing data requires that all data you want to use is in RAM. The larger the amount of data, the more RAM the *XMLInterface* uses!

You might, for example:

- Select data from the XML document.

The source code looks like this:

```
// randomly accesses data via XPath instructions
var tbl:table
XMLInterface.FileName := "D:\MSXML 4.0\books.xml"
// load the XML document for random access into RAM
XMLInterface.openDocument
// select nodes via XPath instruction
// selection starts at the Context node which you entered in the
XMLInterface
// the second parameter is the selection depth for each node
// 0 means that no children will be selected
// the result is passed to a table
tbl := XMLInterface.getNodes("book[title='Midnight Rain']", 1)
XMLInterface.close
```

- Delete existing data from the XML document.

The source code looks like this:

```
// deletes all nodes specified by XPath instructions
XMLInterface.FileName := "D:\MSXML 4.0\books.xml"
// load the XML document for random access into RAM
XMLInterface.openDocument
// delete all book nodes of genre 'Fantasy'
XMLInterface.deleteNodes("book[genre = 'Fantasy']")
XMLInterface.FileName := "D:\MSXML 4.0\tmp.xml"
// write the document to a file
XMLInterface.write
```

```
// remove the document from RAM
XMLInterface.close
```

- Insert new data into the XML document.

The source code looks like this:

```
// inserts new data into the XML document
var tbl:table
XMLInterface.FileName := "D:\MSXML 4.0\books.xml"
// load the XML document
XMLInterface.openDocument
// get an empty table for writing the data to
// depth=1 means that we want to write nodes with children
tbl := XMLInterface.getContainer(1)
// set the parent node for the new data
XMLInterface.setContext("/catalog")
// designate the node to append to the 'catalog' nodes
tbl[1,1] := "book"
// are the attributes of the 'book' node
tbl.createNestedList(4,1)
// explicit namespace
tbl[4,1][1,1] := "xmlns:aa"
tbl[4,1][2,1] := "specAth"
// additional attributes
tbl[4,1][1,2] := "id"
tbl[4,1][2,2] := "bk113"
// child nodes
tbl.createNestedList(5,1)
tbl[5,1][1,1] := "aa:author"
tbl[5,1][2,1] := "specAth"
tbl[5,1][3,1] := "XYZ"
tbl[5,1][1,2] := "title"
tbl[5,1][3,2] := "UNKNOWN"
tbl[5,1][1,3] := "genre"
tbl[5,1][3,3] := "also"
tbl[5,1][1,4] := "price"
tbl[5,1][3,4] := "12,45"
tbl[5,1][1,5] := "publish_date"
tbl[5,1][3,5] := "12.1.02"
tbl[5,1][1,6] := "description"
tbl[5,1][3,6] := "xx0011"
XMLInterface.insertNodes(tbl)
// save the changed document
XMLInterface.FileName := "D:\MSXML 4.0\tmp.xml"
XMLInterface.write
// close the document
XMLInterface.close
```

- Update the XML document.

The source code looks like this:

```
// updates the selected nodes of the document
var tbl:table;
XMLInterface.FileName := "D:\MSXML 4.0\books.xml"
XMLInterface.openDocument;
// select the nodes to be changed
tbl := XMLInterface.getNodes("/catalog/book[title='Midnight Rain']", 1)
// update the values
tbl[5,1][3,3] := "TEST"
// write the changed values
XMLInterface.updateNodes(tbl)
XMLInterface.FileName := "D:\MSXML 4.0\tmp.xml"
XMLInterface.write
XMLInterface.close
```

- Create a new XML document.

The source code looks like this:

```
// creates a new document by calling the method newDocument
var tbl:table;
XMLInterface.newDocument("catalog")
tbl := XMLInterface.getContainer(1)
XMLInterface.setContext("/catalog")
// parent node
tbl[1,1] := "book"
// default namespace
tbl[2,1] := "MyBooks"
// attributes
tbl.createNestedList(4,1)
// explicit namespace
tbl[4,1][1,1] := "xmlns:aa"
tbl[4,1][2,1] := "specAth"
// additional attributes
tbl[4,1][1,2] := "id"
tbl[4,1][2,2] := "bk113"
// child nodes
tbl.createNestedList(5,1)
tbl[5,1][1,1] := "aa:author"
tbl[5,1][2,1] := "specAth"
tbl[5,1][3,1] := "XYZ"
tbl[5,1][1,2] := "title"
tbl[5,1][3,2] := "UNKNOWN"
tbl[5,1][1,3] := "genre"
tbl[5,1][3,3] := "also"
tbl[5,1][1,4] := "price"
tbl[5,1][3,4] := "12,45"
tbl[5,1][1,5] := "publish_date"
tbl[5,1][3,5] := "12.1.02"
tbl[5,1][1,6] := "description"
tbl[5,1][3,6] := "xx0011"
XMLInterface.insertNodes(tbl)
```

```
XMLInterface.FileName := "D:\MSXML 4.0\tmp.xml"
XMLInterface.write
```

Back to [Import Data in XML Format](#)

Access and Traverse Data Randomly

You can extract data in its entirety from an XML document and then **randomly traverse the data**. You might, for example, define the starting point for traversing the structure with the method [selectNodes](#). Then, you could get the next node with the method [getNodeName](#), check for attributes of this node with the read-only attribute [GetNumberAttributes](#), output the names of the attributes and recursively check all children of the selected node, to detect those that meet the condition.

The source code looks like this:

```
// selects the starting nodes and calls the method visitChildren
// recursively for each node
var numberAttributes
XMLInterface.FileName := "D:\Public\XML\books.xml"
// load the XML document to be randomly accessed into RAM
XMLInterface.openDocument
// select some nodes using XPath instructions
XMLInterface.selectNodes("book[genre = 'Computer']")
// define the loop for the selected nodes
while XMLInterface.getNextNode = true
  print XMLInterface.getNodeName
  // check for attributes of the nodes
  numberAttributes := XMLInterface.getNumberAttributes
  for var i := 0 to numberAttributes-1
    // print the data of the attributes
    print XMLInterface.getAttributeName(i)+": "
  +XMLInterface.getAttributeValue(i)
  next
  // check the children of the current node
  VisitChildren
end
// remove the XML document from RAM
XMLInterface.close
```

Back to [Import Data in XML Format](#)

Importing Data from a Database

Import Data from a Database

You can import data from a database into *Plant Simulation* and run your simulation in *Plant Simulation* with this data. You can then write the data resulting from the simulation runs back into the database.

To get the most out of importing data from a database, you have to be familiar with *SQL (Structured Query Language)* instructions. You might, for example, check the web site <http://sqlzoo.net> for more information and consult the documentation that comes with the database you are using. In addition, you have to be familiar with programming in *SimTalk*.

You can:

- [Import Data from an ODBC Database](#)
- [Import Data from an Oracle Database](#)
- [Exchange Data with an SQL Database](#)

Back to [Import and Export Data](#)

Importing Data from an ODBC Database

Import Data from an ODBC Database

To import data from an ODBC database into your *Plant Simulation* simulation model, you will use the *Plant Simulation* object [ODBC](#) . To access several databases at the same time, you can insert several objects of type *ODBC* into your simulation model, each of which communicates with another database.

Note

The example below only runs if 64-bit ODBC drivers are installed on your 64-bit Windows operating system.

You cannot install 64-bit ODBC drivers in parallel to a 32-bit-version of Microsoft Office.

You can also access the same database with several *ODBC* objects. We advise against doing so, as this may result in inconsistent data caused by conflicting commands. It might, for example, happen that one *Method* deletes a data set, while another *Method* attempts to access just this data set. Using a single *ODBC* object ensures that the sequence of instructions conforms to your intentions.

This holds especially true for real databases for which you have to apply any data manipulations, such as read, write delete, etc. with the SQL instruction `commit`, before they take effect.

To import data from an ODBC database into your *Plant Simulation* simulation model you have to:

- [Set the Data Source Up](#)
- [Import Data into Your Simulation Model](#)

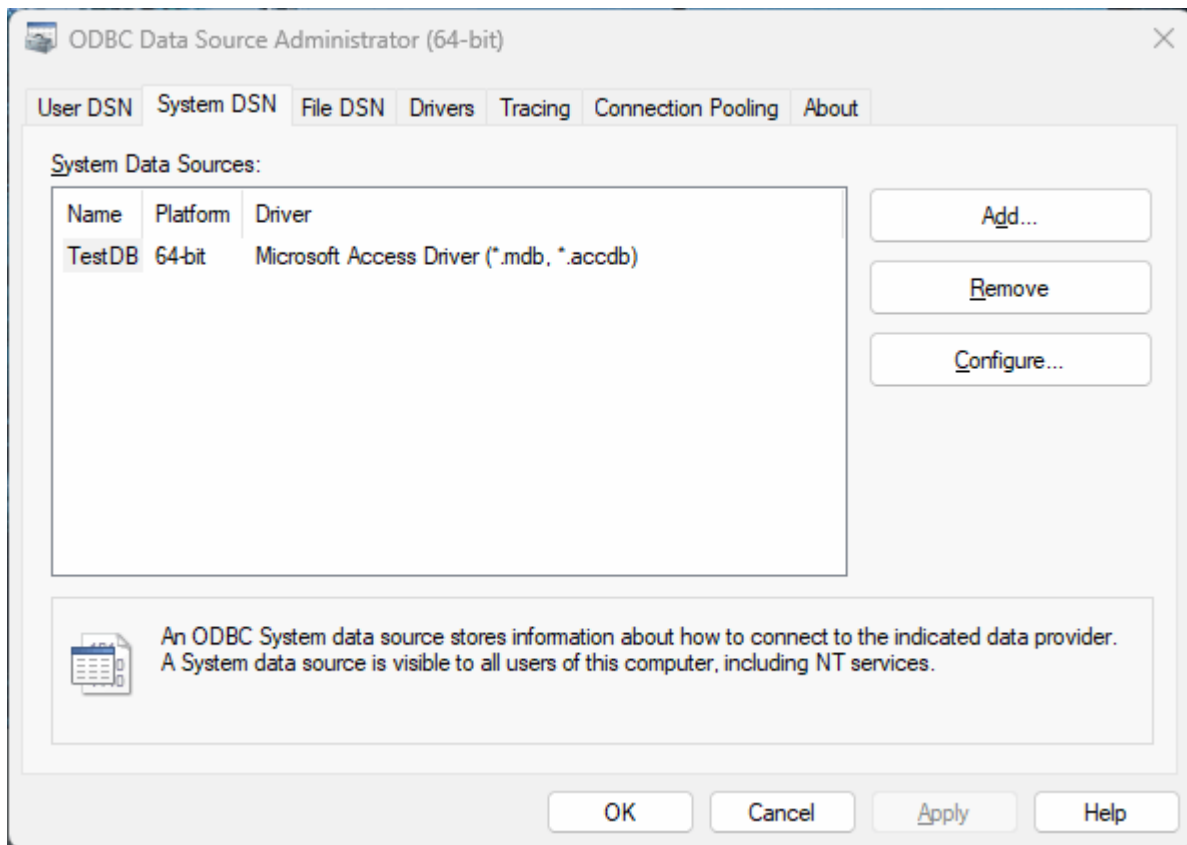
After the simulation runs are finished, you can also export the simulation results back into the database, compare [Export Data to the Database](#).

Back to [Import Data from a Database](#)

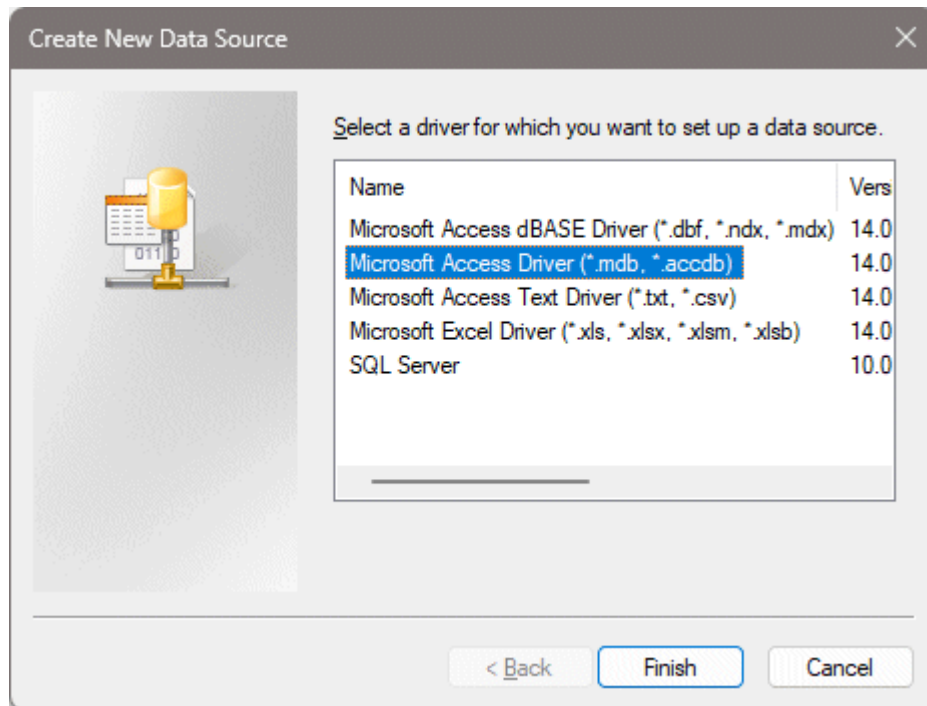
Set the Data Source Up

Before you can import data from an ODBC database into your *Plant Simulation* model, you have to set up the ODBC data source on your computer:

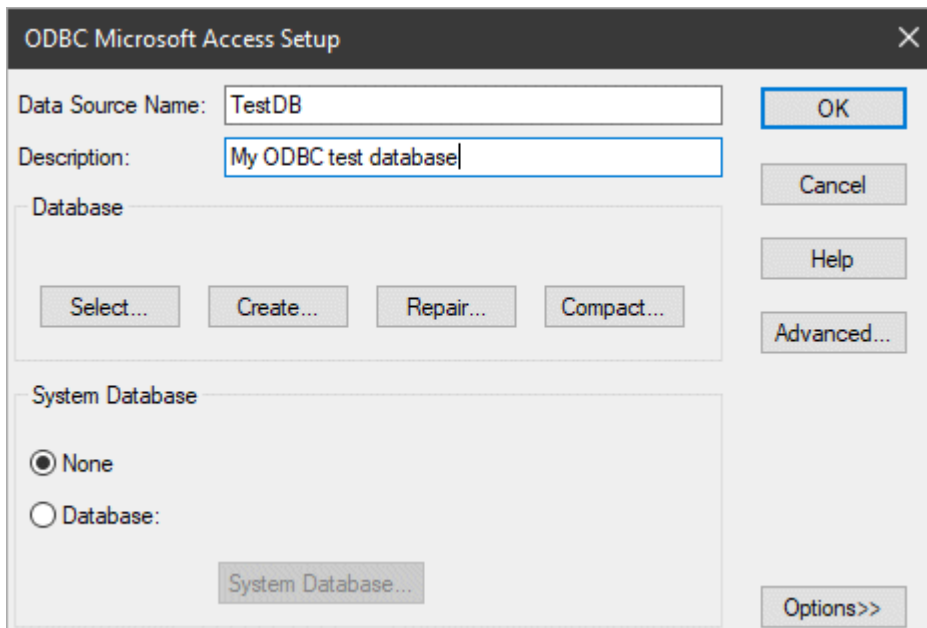
- Type in **ODBC Data Sources (64-bit)** into the text box **Search**.
- Add a new data source in the dialog **ODBC Data Source Administrator**. Depending on who will use the database you can accomplish this on the tab **User DSN** or on the tab **System DSN**. As a rule you will add the data source on the tab **System DSN**.



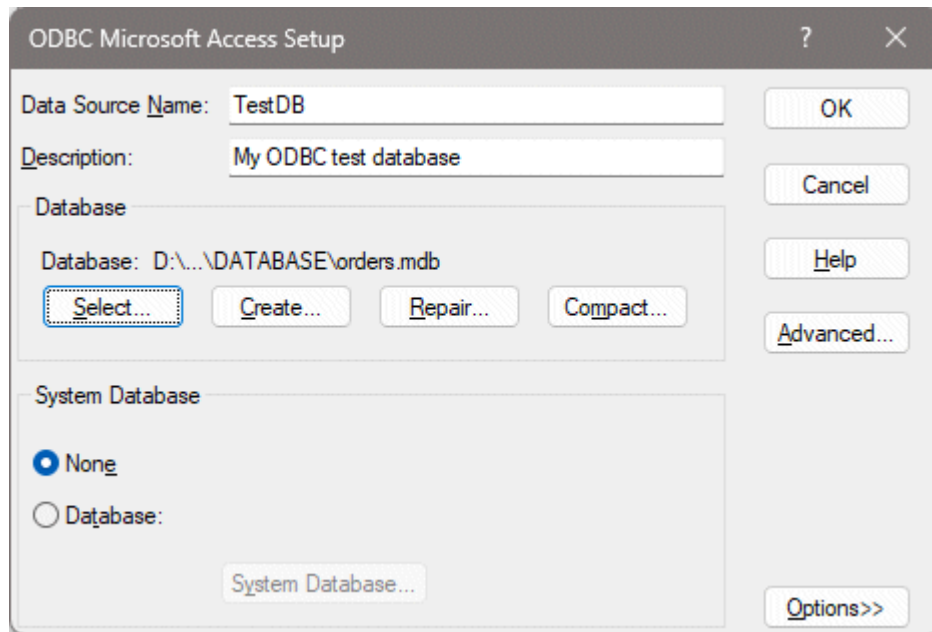
- Click **Add** on the tab **System DSN**.
- In the dialog **Create New Data Source** select the ODBC driver for the data source you would like to use. In our example we selected the **Microsoft Access Driver**. Click **Finish**.



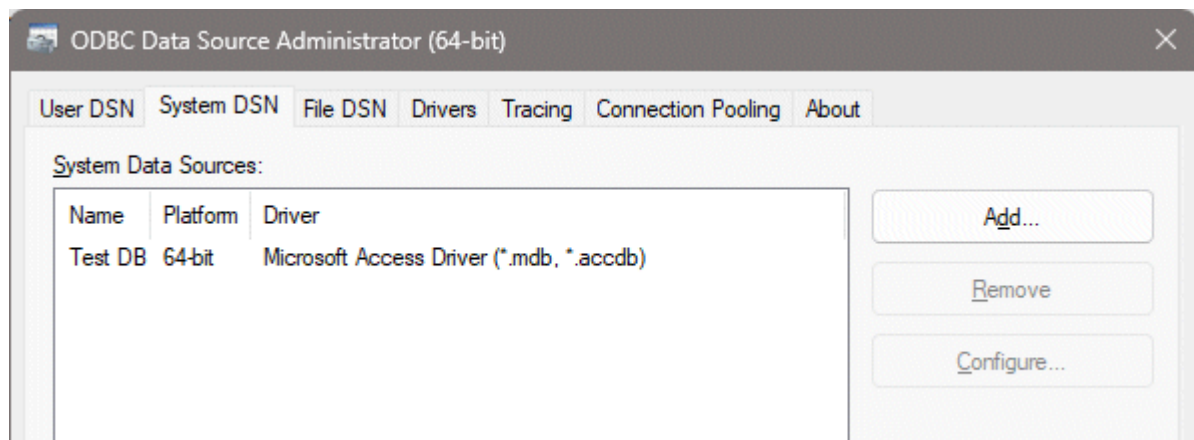
- Click **Select** in the dialog **ODBC Microsoft Access Setup** and select the database in the dialog **Select Database**. Type in the **Data Source Name**. If you want to, you can also add a **Description** of the database. As *Plant Simulation* addresses the database with this name, mind the *Plant Simulation* conventions for the **Name**!



- Click **Select** and select the database to which you want to connect.



After successfully concluding this action, the dialog **ODBC Data Source Administrator** shows the **Data Source**, which we added.

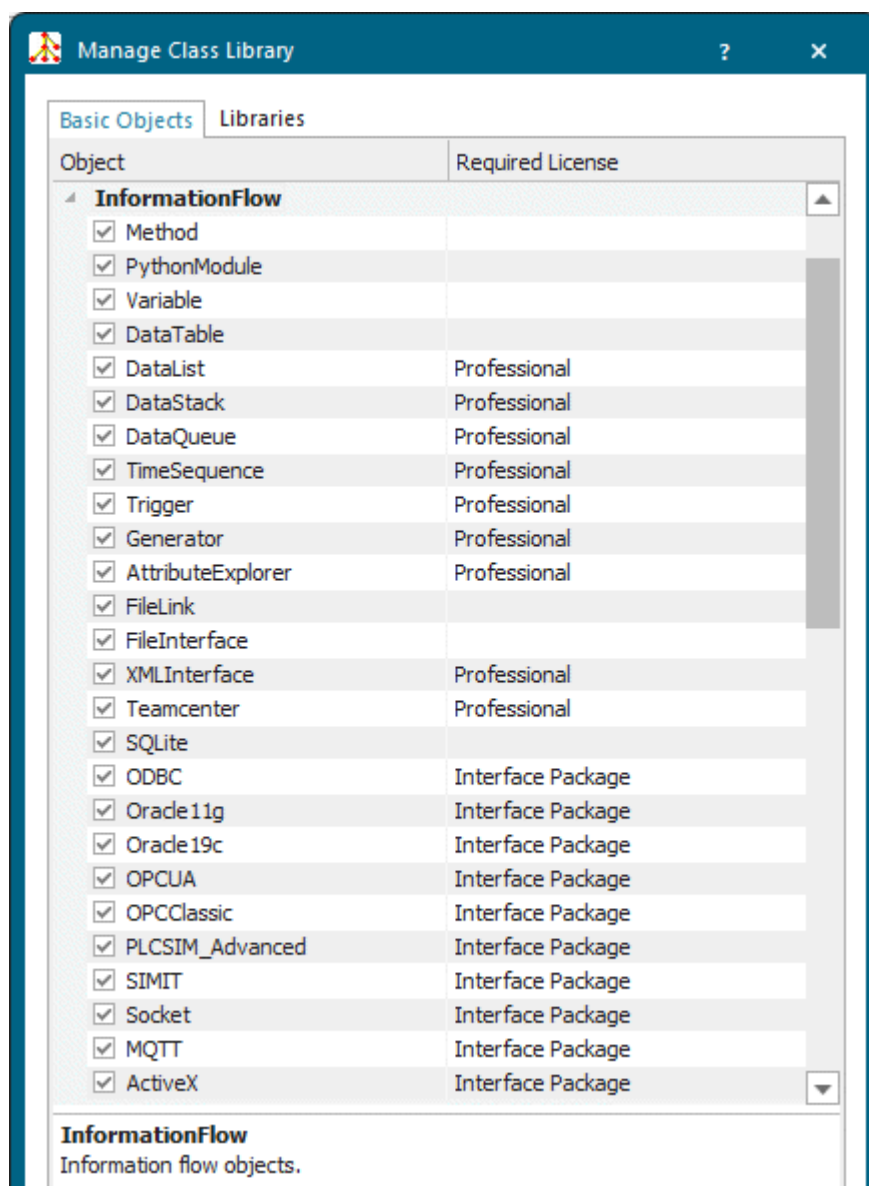


Back to [Import Data from an ODBC Database](#)

Import Data into Your Simulation Model

After setting-up the ODBC data source, you have to insert the object *ODBC* into your *Plant Simulation* simulation model. This object establishes the connection with the database and enables you to import data from it. You can write this data into *Plant Simulation* tables, change them there, use them in your simulation runs, and re-export the changed data back into the database.

- To add the object **ODBC**  to the toolbar **Information Flow** of your simulation model, click **Manage Class Library > Basic Objects > InformationFlow** on the **Home** ribbon tab. 



Click the object on the toolbar **Information Flow** and insert it into your simulation model.

In our example we inserted the following objects into our simulation model:

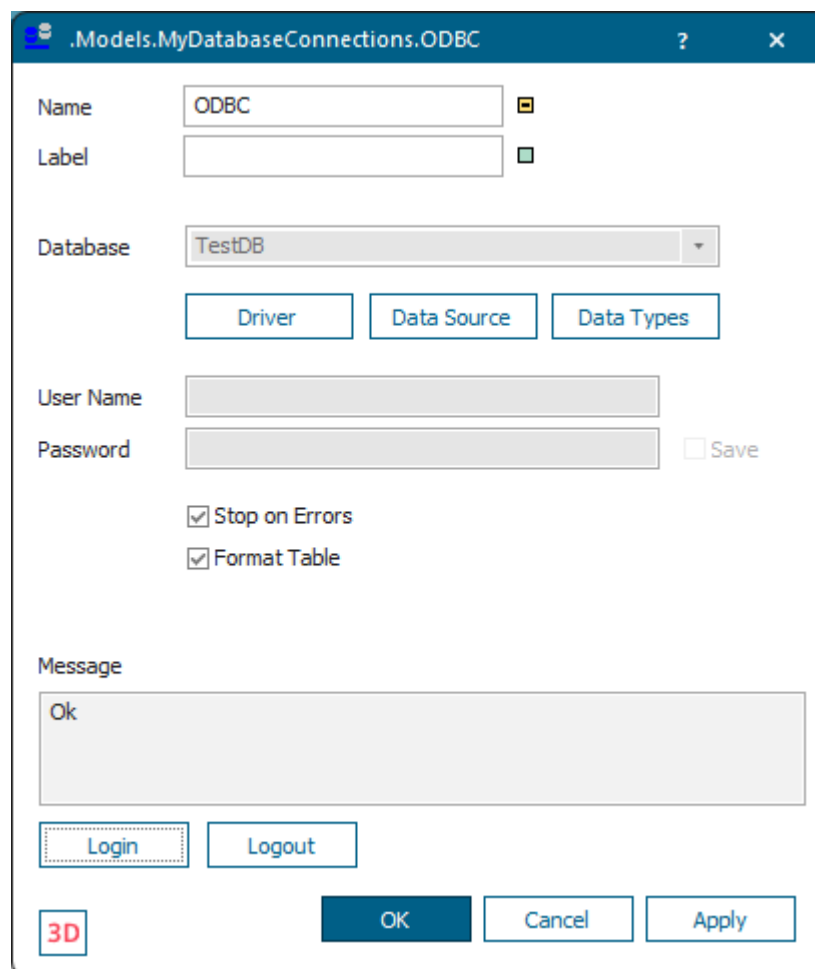
- An *ODBC* object, which controls the communication with the database.
- A *Method* for reading data from the database.
- A *Method* for writing data to the database.
- A *Plant Simulation DataTable* into which we import the data, and from which we export it back to the database.



- Double-click the object *ODBC* and type the name of the database into the text box **Database**. In our example we entered **TestDB**. This is the same name that we entered into the dialog **ODBC Microsoft Access Setup** above.

When you use a database with user management, such as SQL-Server, Oracle, etc., you also have to enter a **User name** and a **Password**. To apply your changes, click **Apply**.

To log on to the database, click **Login**.



When all your settings worked correctly, *Plant Simulation* dims the box with the name of the database and shows **Ok** in the box **Message**. When *Plant Simulation* encountered problems, it shows an error message describing the problem.

- Then, insert a *Method* into your simulation model. Within this *Method* you program, which data will be imported and what happens to this data.

Reading and writing data only works, when *Plant Simulation* is connected to the database. For this reason the methods **login** and **logout** frame the database operation proper in the instructions.

The source code looks like this:

```
ODBC.login("TestDB", "", "")  
// database operation  
ODBC.logout
```

- Now you can read data from the database and write the results of the database query into an *Plant Simulation* table. Instead, you could also write the data into a local variable.

You'll always start the query with the command **sql**. Then, define the target, in our example this is the *Plant Simulation* table named **Orders**. Next, you'll enter SQL standard queries within quotation marks. The syntax you have to use depends on your database system, consult the documentation that came with your database. The ODBC driver of the database determines the range of commands you can enter.

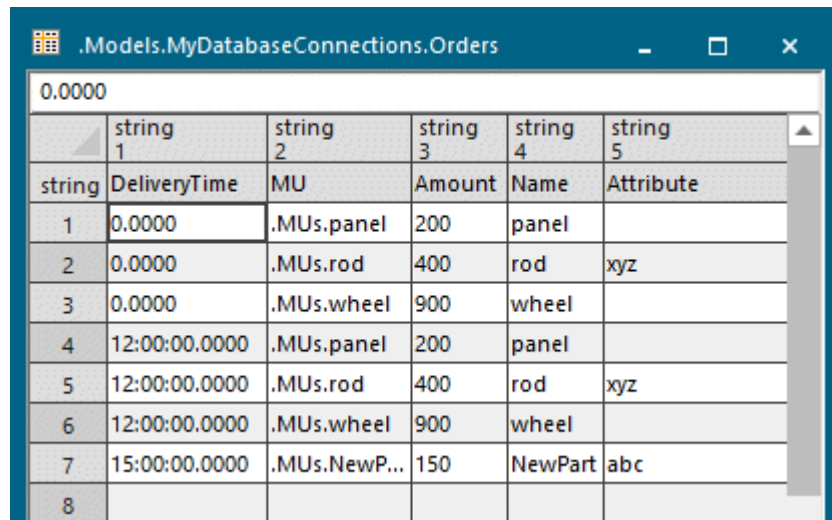
In our example we will read the entire contents of the table **Orders2** from our Access database and write it into the *Plant Simulation* table **Orders**.

DeliveryTime	MU	Amour	Name	Attribute
0.0000	.MUs.panel	200	panel	
0.0000	.MUs.rod	400	rod	xyz
0.0000	.MUs.wheel	900	wheel	
12:00:00.0000	.MUs.panel	200	panel	
12:00:00.0000	.MUs.rod	400	rod	xyz
12:00:00.0000	.MUs.wheel	900	wheel	
15:00:00.0000	.MUs.NewPart	150	NewPart	abc

The source code looks like this:

```
ODBC.login("TestDB","","")
ODBC.sql(Orders, "select * from Orders2")
ODBC.logout
```

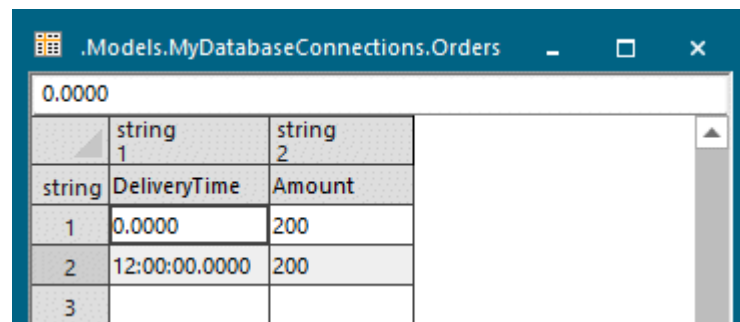
If you want to format the columns of the target table in *Plant Simulation* according to the formatting in the database while reading the data, make sure that **Format table** ☒ **Format table** in the dialog of the object *ODBC* is checked. This only applies, when *Plant Simulation* provides its own formats, which correspond to the formats in the database. *Plant Simulation* does, for example, not provide a counterpart for the typical date format in Oracle. The documentation of your database provides information about filters for changing formats during the query.



	string 1	string 2	string 3	string 4	string 5	string 6
	DeliveryTime	MU	Amount	Name	Attribute	
1	0.0000	.MUs.panel	200	panel		
2	0.0000	.MUs.rod	400	rod	xyz	
3	0.0000	.MUs.wheel	900	wheel		
4	12:00:00.0000	.MUs.panel	200	panel		
5	12:00:00.0000	.MUs.rod	400	rod	xyz	
6	12:00:00.0000	.MUs.wheel	900	wheel		
7	15:00:00.0000	.MUs.NewP...	150	NewPart	abc	
8						

- You can use and manipulate this data in *Plant Simulation* in a number of ways. When working with large amounts of data, we recommend to use SQL queries with filters, as these are oftentimes considerably faster than searching large *Plant Simulation* tables.

In our example the query for DeliveryTime and Amount of all parts of type panel yields this result in our *Plant Simulation* table.



	string 1	string 2	string 3
	DeliveryTime	Amount	
1	0.0000	200	
2	12:00:00.0000	200	
3			

The source code looks like this:

```
ODBC.login("TestDB","","")
ODBC.sql(Orders, "select DeliveryTime, Amount from Orders2 where MU =
'.MUs.panel'")
ODBC.logout
```

Back to [Import Data from an ODBC Database](#)

Export Data to the Database

You can also export selected results of your simulation runs back to the ODBC database. For this, you use the method `sql` and a number of SQL instructions.

In our example we add a new row to our Access table *Orders2* using the SQL instruction `insert into`.

The source code looks like this:

```
ODBC.login("TestDB","","")
ODBC.sql("insert into Orders2 values ('15:00:00.0000', '.MUs.NewPart',
'150', 'NewPart', 'abc')")
ODBC.logout
```

Orders2				
DeliveryTime	MU	Amour	Name	Attribute
0.0000	.MUs.panel	200	panel	
0.0000	.MUs.rod	400	rod	xyz
0.0000	.MUs.wheel	900	wheel	
12:00:00.0000	.MUs.panel	200	panel	
12:00:00.0000	.MUs.rod	400	rod	xyz
12:00:00.0000	.MUs.wheel	900	wheel	
15:00:00.0000	.MUs.NewPart	150	NewPart	abc
*				

Note

As SQL does not provide a single statement for adding the contents of a row or of the entire table to the database, you have to type the contents of each cell into the *Method*.

To add data to an existing data set, thus updating the database, use the SQL instruction update:

The source code looks like this:

```
ODBC.login("TestDB","","")
ODBC.sql("update Orders2 set Attribute = 'xyz' where Name = 'rod'")
ODBC.logout
```

Orders2				
DeliveryTime	MU	Amour	Name	Attribute
0.0000	.MUs.panel	200	panel	
0.0000	.MUs.rod	400	rod	xyz
0.0000	.MUs.wheel	900	wheel	
12:00:00.0000	.MUs.panel	200	panel	
12:00:00.0000	.MUs.rod	400	rod	xyz
12:00:00.0000	.MUs.wheel	900	wheel	
15:00:00.0000	.MUs.NewPart	150	NewPart	abc
*				

Back to [Import Data from an ODBC Database](#)

Import Data from an Oracle Database

In principle you will use the object *Oracle11g/Oracle19c* in pretty much the same way as you do the object *ODBC*. First, establish a connection with a database instance, which exists on the Oracle Server. This database instance determines the name of the database, which you type into the object *Oracle11g*. Microsoft Windows does not provide Oracle settings.

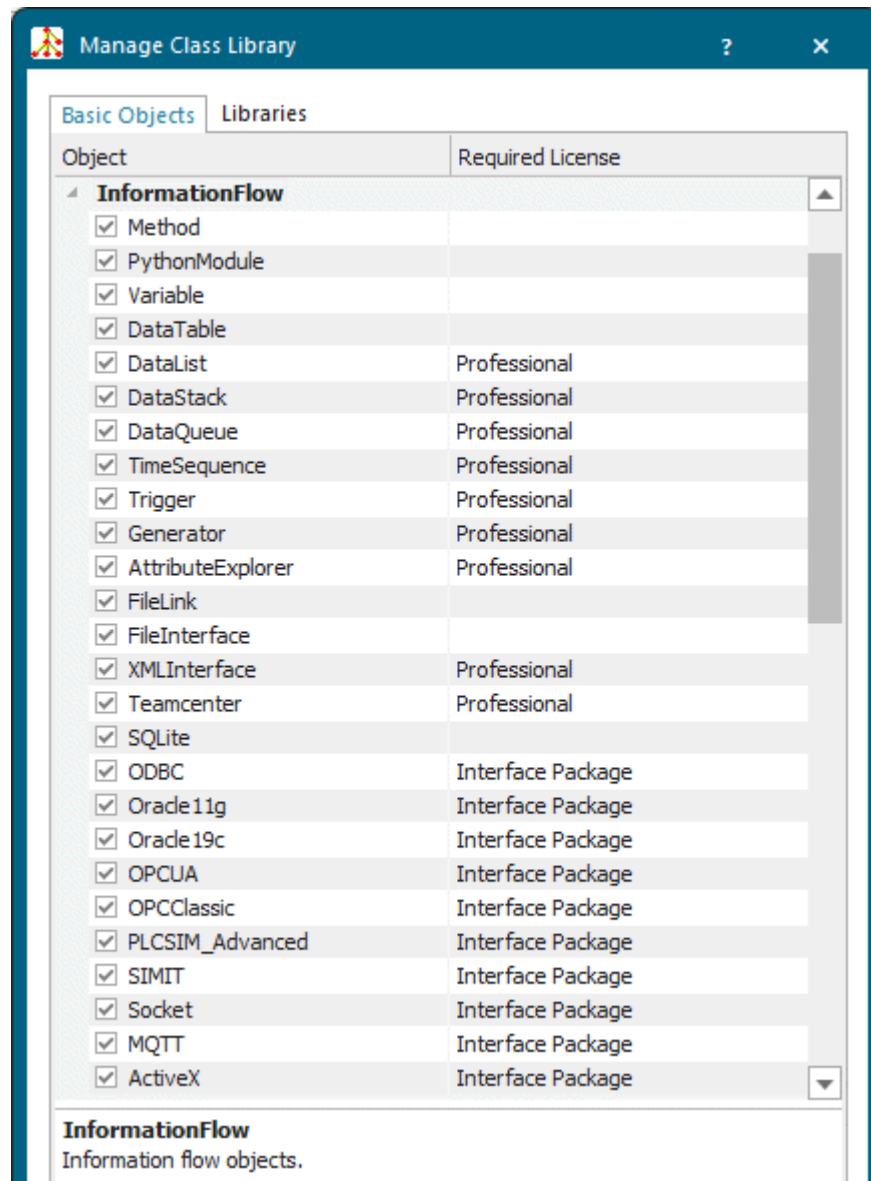
If the Oracle Server is not installed on the same computer on which you execute *Plant Simulation*, you have to install an Oracle Client on that computer, which establishes the connection to the server. If the Oracle Client is not included in your Oracle Installation, contact your Oracle provider to obtain it.

If you have to handle large amounts of data, the object *Oracle11g* performs better and provides more commands than the object *ODBC*.

After you established the connection to an external Oracle database server within the computer network that your local computer can access, insert the object *Oracle11g* into your *Plant Simulation* simulation model. It connects to the database and imports data from it. You can write this data into *Plant Simulation* tables, change them there and use them in your simulation runs and re-export the changed data back into the database.

To add **Oracle11g**  to the toolbar **Information Flow** of your simulation model, click  **Manage Class Library** > **Basic Objects** > **InformationFlow** > **Oracle11g** on the **Home** ribbon tab.

Click the object on the toolbar **Information Flow** and insert it into your simulation model.

**Note**

You can also use ODBC together with Oracle! Then, you do not have to install an Oracle Client. Be aware that the SQL instructions, which you use, are not necessarily compatible with the ODBC version. This may cause problems if, for example, you want to switch between an Oracle database and an Access-database for test purposes. In this case, we recommend to use ODBC for all databases.

Back to [Import Data from a Database](#)

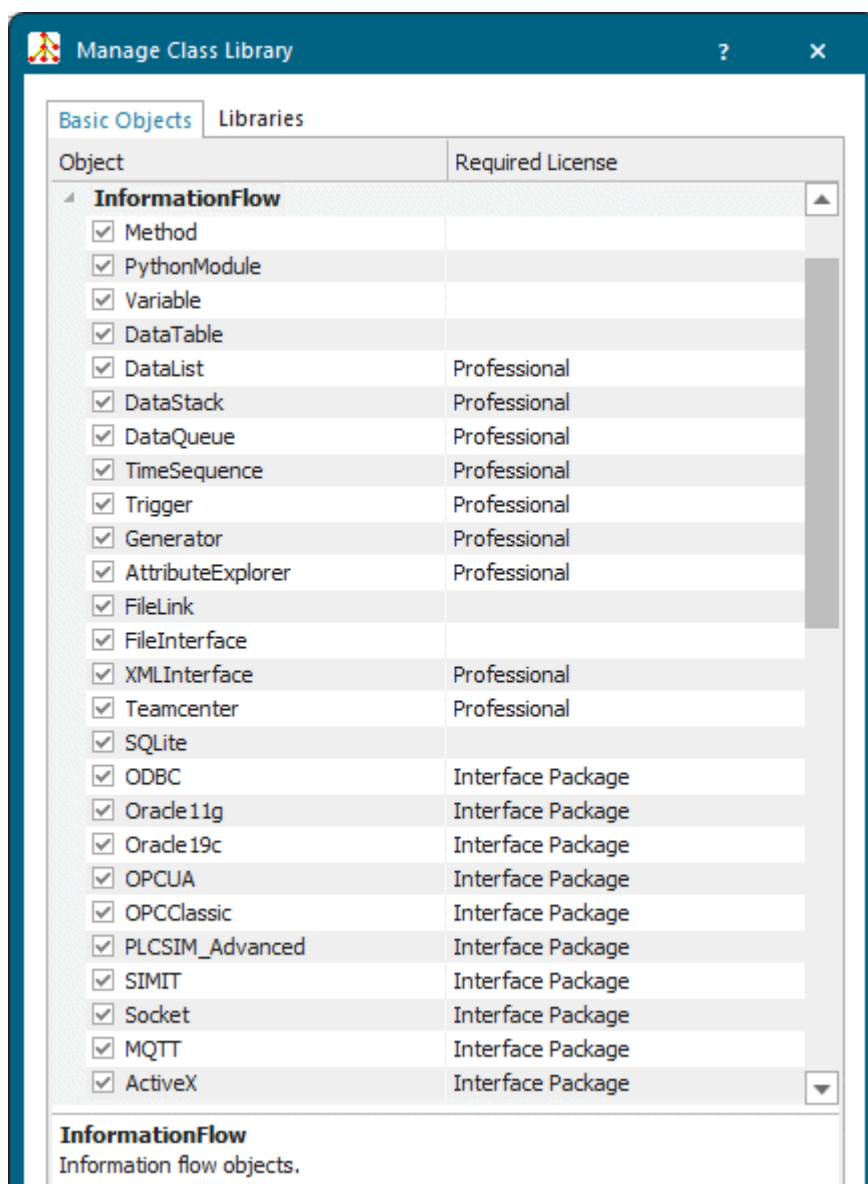
Exchanging Data with an SQL Database

Exchange Data with an SQL Database

Plant Simulation can exchange data with an SQL database using the object **SQLite**.

To add the object **SQLite**  to the toolbar **Information Flow** of your simulation model, click  **Manage Class Library** > **Basic Objects** > **InformationFlow** > **SQLite** on the **Home** ribbon tab.

Select the object on the toolbar **Information Flow** and insert it into your simulation model.



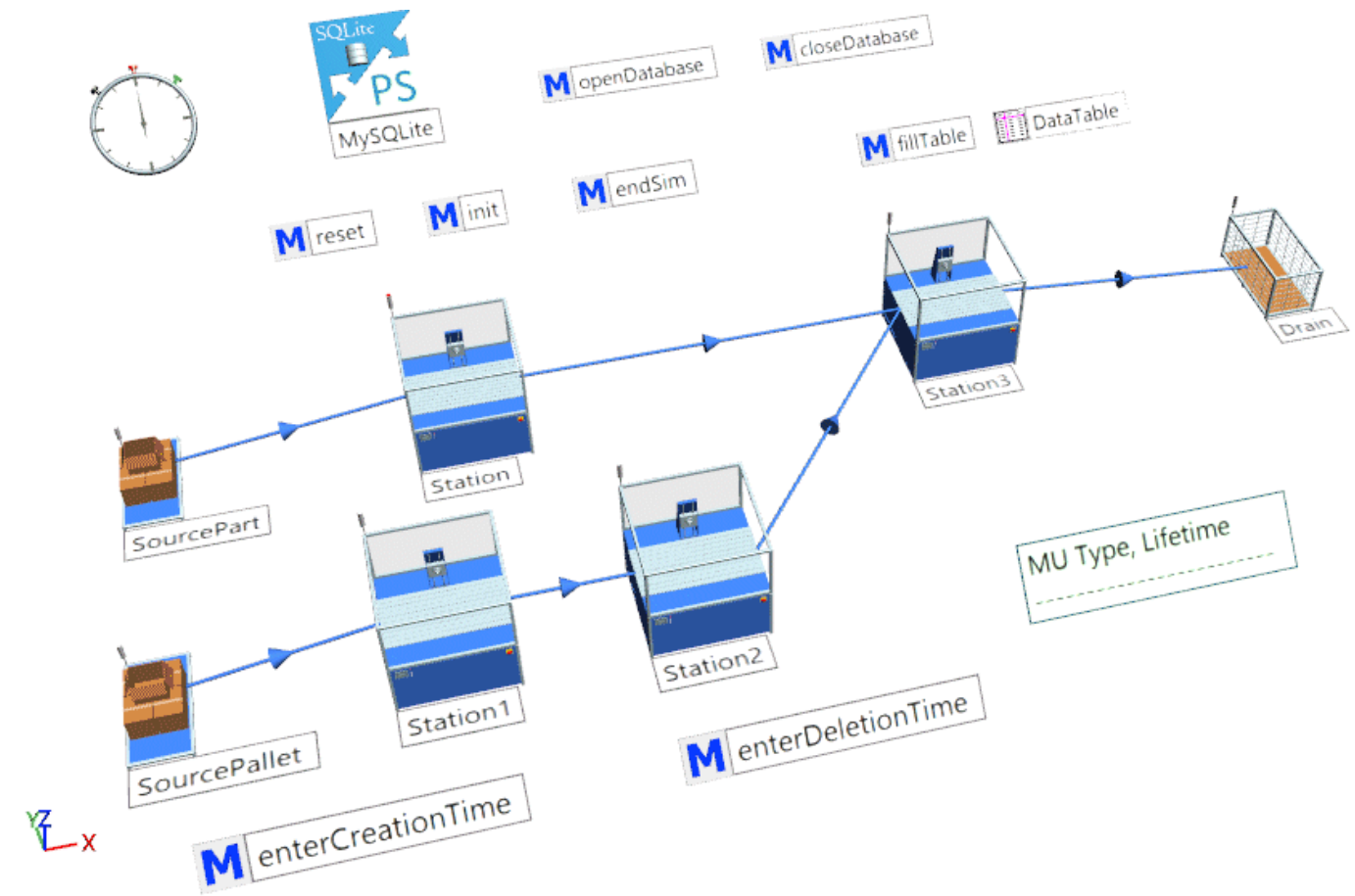
This sample model demonstrates how to use the object *SQLite* to establish a connection between *Plant Simulation* and an SQL database.

- The object *MySQLite* uses a database which is stored in main memory instead of in a file on your computer's hard disk.
- The method *openDatabase* opens the database and creates a table in the database using SQL statements.
- The objects *SourcePart* and *SourcePallet* produce different parts that pass through the plant. When the parts leave the respective *Source* objects, the method *enterCreationTime* records the part type and the creation time in the database.
- Once the parts reach the *Drain*, the method *enterDeletionTime* records the time at which the parts leave the plant.
- The simulation runs for six days. At the end of the simulation run the *endSim* method computes the average lifetime of the parts and the method *fillTable* writes these results into a *Plant Simulation* table.
- The method *closeDatabase*, which is called by the *endSim* method, closes the database.

Proceed as follow to create the model:

- **Configure the Connection with the SQL Database**
- **Configure the Material Flow Through the Facility**
- **Import the Simulation Results From the Database and Show Them**

The finished simulation model looks like this:



Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category > Information Flow**, the **Topic > SQLite Introduction**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.

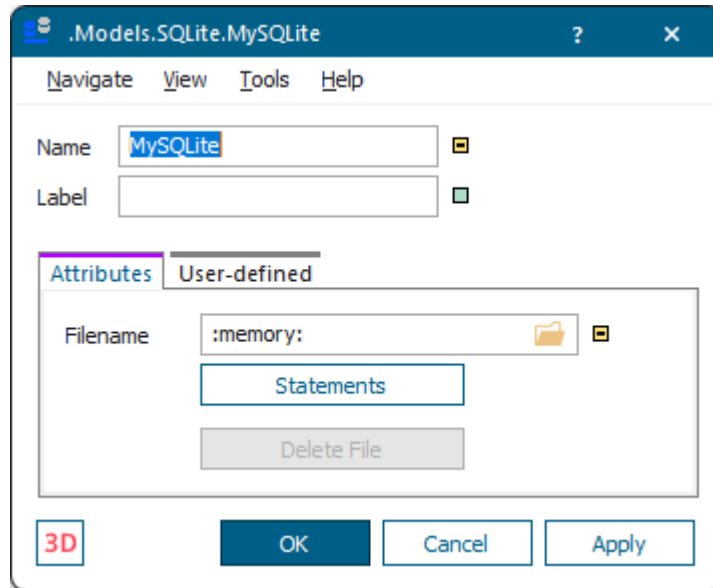
Back to [Import Data from a Database](#)

Configure the Connection with the SQL Database

Proceed as follows to establish a connection with an SQL database with the object *SQLite*.

- Insert the object *SQLite* into the simulation model. We named it *MySQLite* in our model. We used the default setting **:memory:**, which means that the database will be stored in your computer's main memory instead of in a file on the hard disk. This greatly improves performance as compared to saving the file on disk. Be aware though that all data will be gone for good once you close *Plant Simulation* or if *Plant Simulation* crashes.

If you do want to use the data later on, enter the file name of the database from which you want to import data or to which you want to export data.



- Program the method to open the database. We named our method *openDatabase* and entered the following source code:

```
// called by the init method
MySQLite.open           // opens the database
MySQLite.exec("CREATE TABLE MUTrace (MUName TEXT PRIMARY KEY, MUType
TEXT, StartTime REAL, EndTime REAL)")
// creates the table MUTrace in the database with four columns containing
the
// name of the part, the type of the part, the start time, and the end
time
```

- Program the method to close the database. We named our method *closeDatabase* and entered the following source code:

```
// called by the endsim method
MySQLite.close // closes the database
```

- We programmed the *reset* method, which is called when you click the **Reset Simulation** button in the *EventController*, to delete the contents of the *DataTable* and the result numbers from the *Comment*:

```
DataTable.delete
DataTable.closeDialog
Comment.Text := "MU Type,
Lifetime"+strChr(13)+strChr(10)+"-----"
```

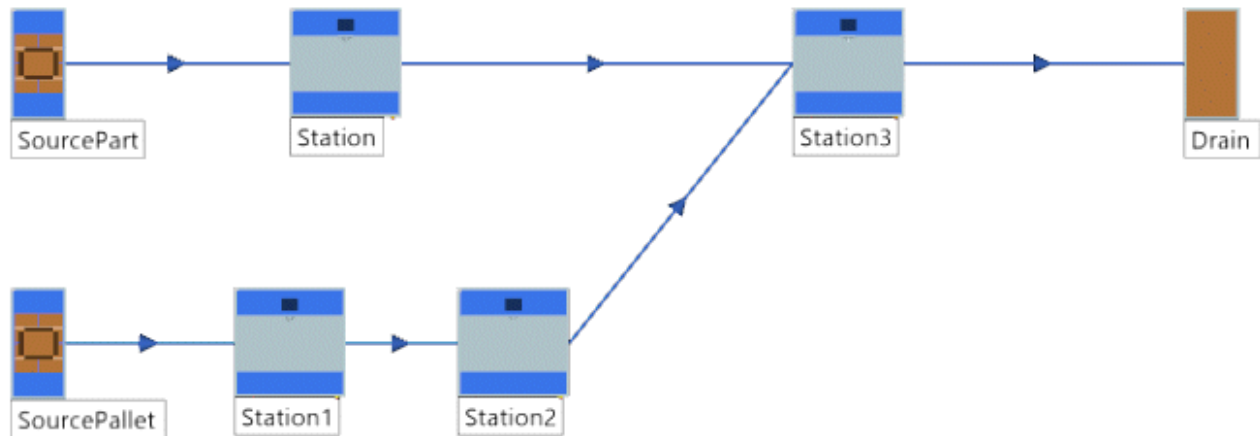
- We programmed the *init* method to show **Processing...** in the *Comment* object and to open the database:

```
Comment.Text := "Processing..."
openDatabase
```

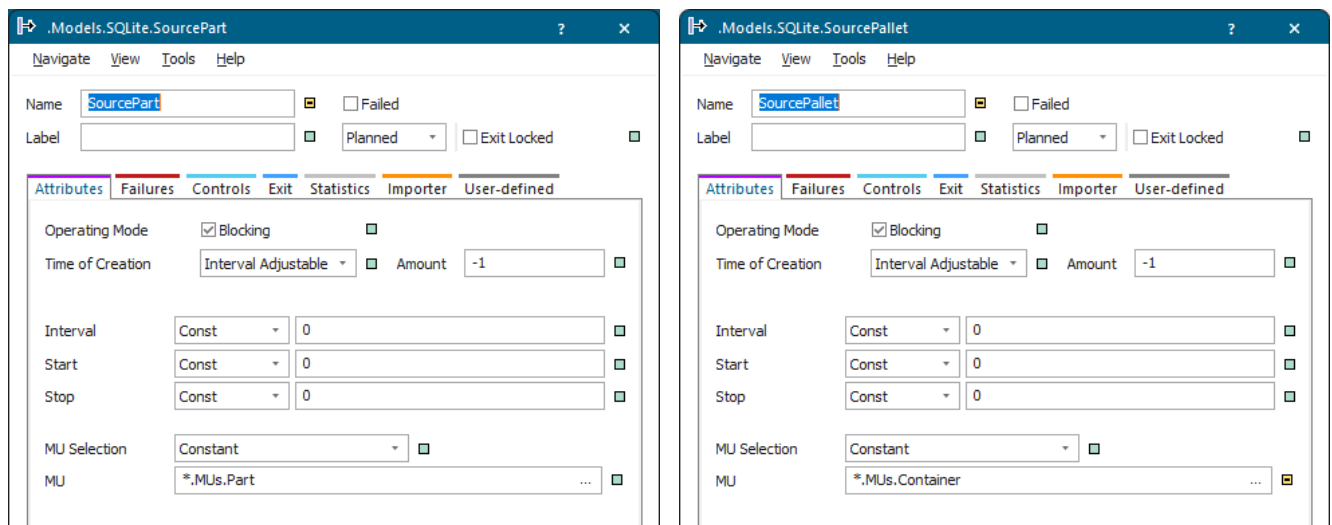
Back to Exchange Data with an SQL Database

Configure the Material Flow Through the Facility

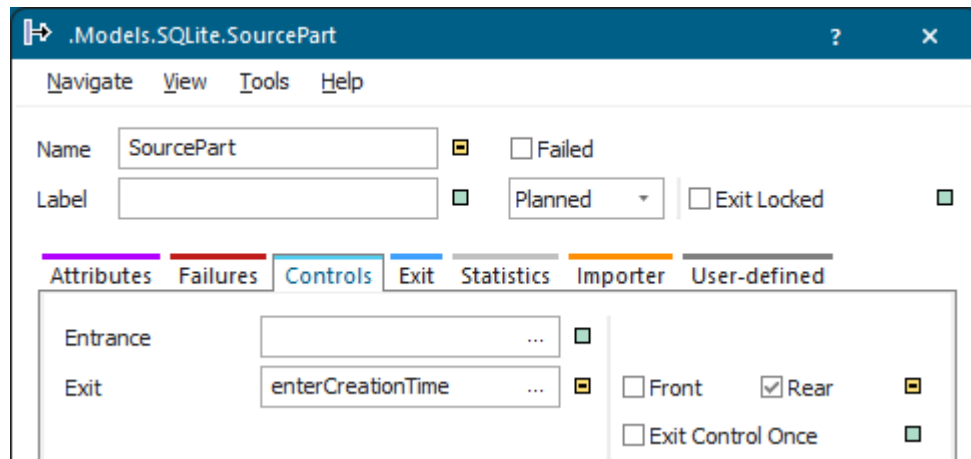
Proceed as follows to configure the material flow through the facility.



- Inserted two *Source* objects into the model and configure them so that they produce two different types of parts. In our model the *SourcePart* produces parts of type *Part*. The *SourcePallet* produces parts of type *Container*.



Next, we selected the same rear-triggered **Exit Control** for both *Sources*.



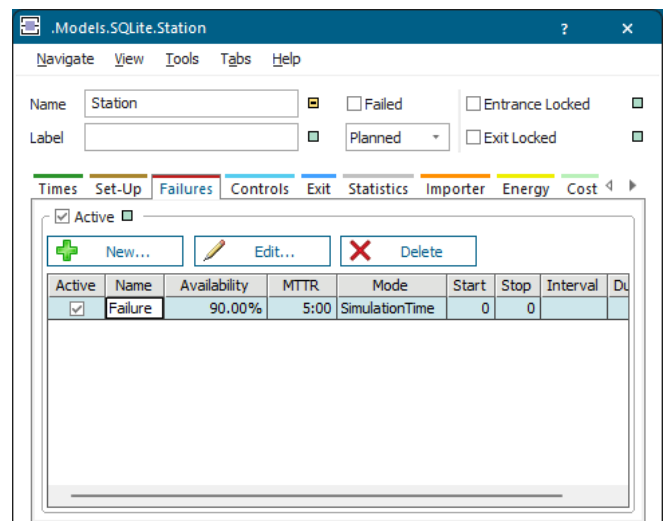
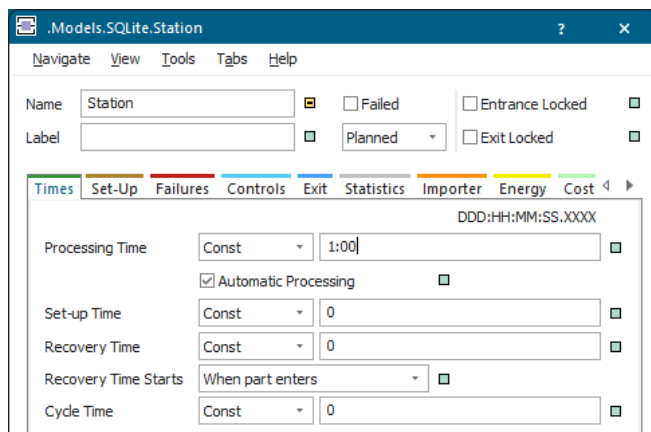
We entered the following source code into the method *enterCreationTime*. It inserts the times at which the parts were created, their type, and the start time into the internal database table named MUTrace. It then binds the string values and real values to the SQL-statement and executes the SQL-statement, thus writing the values to the database.

```

MySQLite.prepare("INSERT INTO MUTrace (MUName, MUType, StartTime) VALUES
(?1, ?2, ?3)")
MySQLite.bindString(1, obj_to_str(@))
MySQLite.bindString(2, @.name)
MySQLite.bindReal(3, EventController.simTime)
MySQLite.step

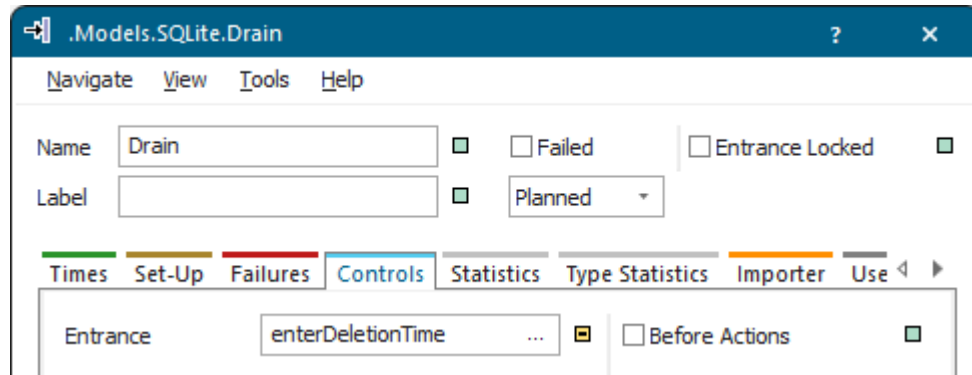
```

- Insert the stations which process the parts. We inserted four *Stations* with these settings:



- Insert the *Drain*, which removes the parts from the plant.

We entered the method *enterDeletionTime* as the **Entrance Control** and left all other settings unchanged.



We entered the following source code into the method. It updates the database table MUTrace with the time at which the parts were deleted.

```
MySQLite.prepare("UPDATE MUTrace set EndTime = ?1 WHERE MUName = ?2")
MySQLite.bindReal(1, EventController.simTime)
MySQLite.bindString(2, obj_to_str(@))
MySQLite.step
```

Back to [Exchange Data with an SQL Database](#)

Import the Simulation Results From the Database and Show Them

To show the simulation results in our simulation model after the simulation run is finished, you can import data of interest from the database. In our sample model we calculated the average lifetime of the parts and wrote these results into a *Plant Simulation* table and into an object of type **Comment**

To do so, we inserted a *DataTable* into our simulation model and assigned the data type *string* to column 1 and 2 and the data type *time* to column 3 and 4. We also inserted an object of type *Comment*.

We then inserted an *endSim* method and entered the following source code. It computes the average lifetime of the part types *Part* and *Container* and writes that result into the *Comment* object. It then calls the method *fillTable*, which in turn writes the lifetime of the individual parts into the *DataTable*:

The source code looks like this:

```
var str := "MU Type,
Lifetime"+strChr(13)+strChr(10)+"-----"
MySQLite.prepare("SELECT MUType, avg(EndTime-StartTime) FROM MUTrace GROUP
BY MUType")
while MySQLite.step
    str := str+strChr(13)+strChr(10)+MySQLite.getColumnString(0)+",
"+to_str(MySQLite.getColumnReal(1))
Comment.Text := str
fillTable      // name of the method that writes data to the DataTable
closeDatabase
```

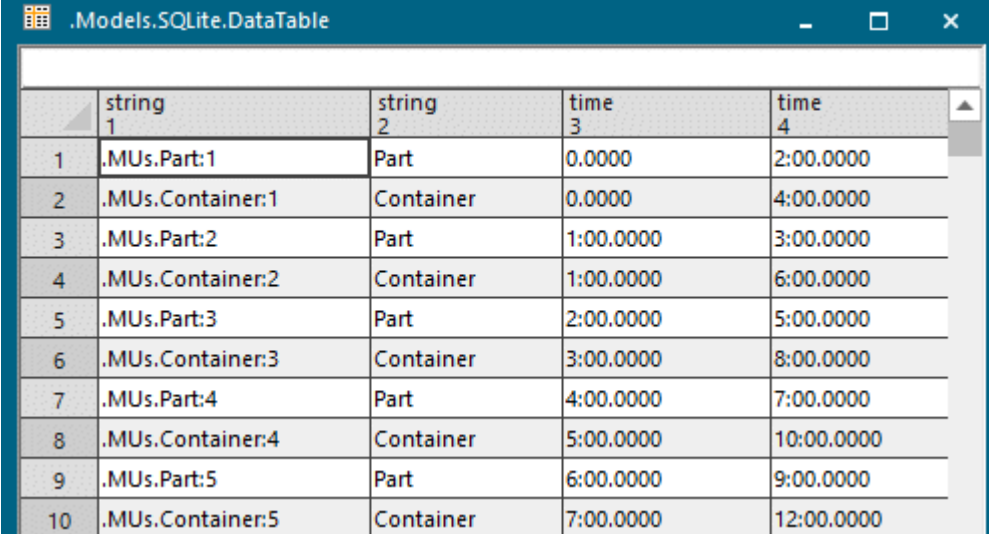
MU Type, Lifetime

Container, 347.499225422239
Entity, 190.386119162779

To write the lifetime of the individual parts into the *DataTable*, we programmed the method *fillTable*.

The source code looks like this:

```
// called by the endSim method
MySQLite.prepare("SELECT * FROM MUTrace")
for var column := 1 to MySQLite.getColumnCount
  var row := 1
  while MySQLite.step
    switch column
    case 1,2 // column 1 and 2 of data type string
      DataTable[column,row] := MySQLite.getColumnString(column - 1)
    else // column 3 and 4 of data type time
      DataTable[column,row] := MySQLite.getColumnReal(column - 1)
    end
    row += 1
  end
end
next
DataTable.opendialog
```



	string 1	string 2	time 3	time 4
1	.MUs.Part:1	Part	0.0000	2:00.0000
2	.MUs.Container:1	Container	0.0000	4:00.0000
3	.MUs.Part:2	Part	1:00.0000	3:00.0000
4	.MUs.Container:2	Container	1:00.0000	6:00.0000
5	.MUs.Part:3	Part	2:00.0000	5:00.0000
6	.MUs.Container:3	Container	3:00.0000	8:00.0000
7	.MUs.Part:4	Part	4:00.0000	7:00.0000
8	.MUs.Container:4	Container	5:00.0000	10:00.0000
9	.MUs.Part:5	Part	6:00.0000	9:00.0000
10	.MUs.Container:5	Container	7:00.0000	12:00.0000

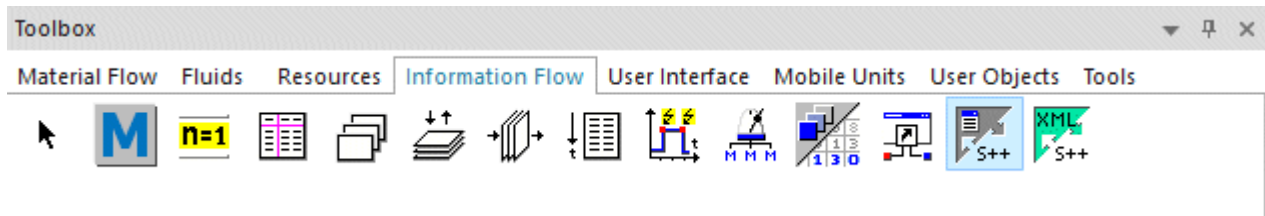
Back to [Exchange Data with an SQL Database](#)

Import or Export Data in ANSI Format

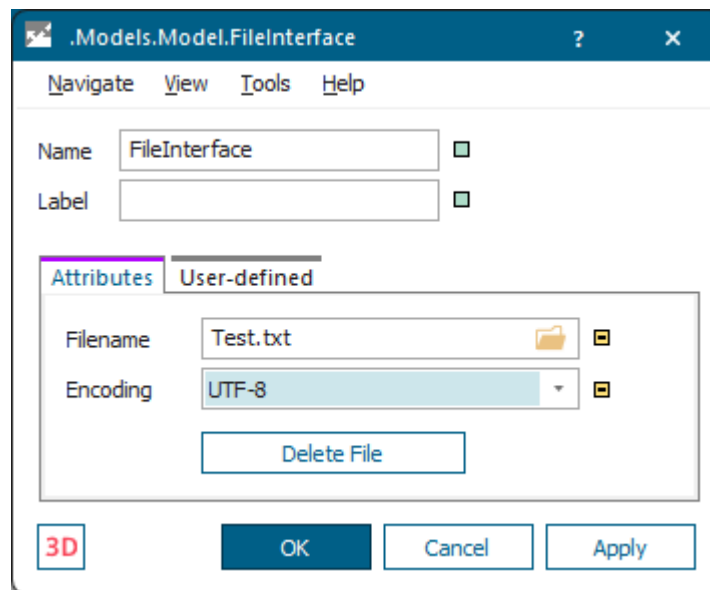
You can import data for a simulation run from a text file with the [FileInterface](#) . You can also export data directly into a text file with the *FileInterface*: You can write log files of your simulation runs or record

statistics data, etc. The *FileInterface* only processes ASCII characters, i.e., letters, numbers and special characters. It provides a number of *methods* to move within the file.

You can insert the *FileInterface* into your simulation model from the folder **InformationFlow** in the *Class Library* or from the toolbar **Information Flow** in the *Toolbox*.



Enter the **Filename [FileInterface]** of the text file, which the *FileInterface* opens, if you want to import data. When you export data, it specifies the name of the file, which it saves.



The method **readLn** opens the file and reads a single line from it. It then increases the internal line counter by one and closes the file again.

The *FileInterface* transforms the contents of the line it read into a *string*. You can manipulate these *strings* with the **Functions for Editing Variables of Data Type String**, such as **strCopy**, **strOmit**, **strLen** and with the **Functions for Converting Data Types**, such as **str_to_num**, **str_to_time** etc.

Calling the method **readLn** again and again moves to the respective next line in the text file. The method **goToLine** moves to the line within the text file, which you specify as an *integer* value. You can then import that line with the method **readLn**. To access the same file several times in a row, we advise to open the file beforehand, as that will increase access speed. If you do not need a file any longer, close it.

Note

The *FileInterface* can keep ten text files open at any one time.

When writing data, for example with the method `write`, the *FileInterface* opens the file and sets its internal line counter to the end of the file by calling the method `goBottom`. Then, the *FileInterface* saves the data to the file and closes it. The *FileInterface* always adds new data to the end of the file, it does not overwrite any existing data.

To access the same file several times in a row, we advise to open it beforehand, as it buffers the data before saving it. The *FileInterface* saves data before the next reading access of the file or when closing it.

Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.

Back to [Import and Export Data](#)

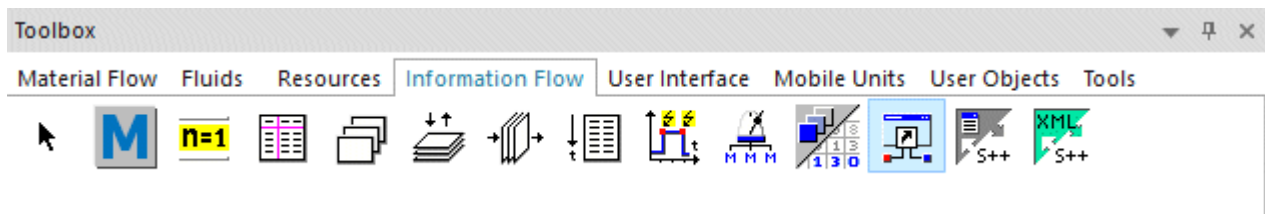
Displaying Data Stored in an External Program

Display Data Stored in an External Program

Provides information on how to display data stored in an external program using *Plant Simulation*.

The **FileLink** enables you to place a link, i.e., a shortcut to a file, into a *Frame* within *Plant Simulation*. This feature enables you to display and edit data stored in external programs directly within your simulation model.

You can insert the *FileLink* into your simulation model from the folder **InformationFlow** in the *Class Library* or from the toolbar **Information Flow** in the *Toolbox*.



Drag a file from the *Desktop* or the *Windows Explorer* into a *Plant Simulation Frame* and drop it there. The *FileLink* creates a link to the file in that *Frame* and places the icon of the application, which is assigned to the specific file type, in the *Frame*. By default, *Plant Simulation* enters the entire path to the file into the text boxes **Label** and **File** name of the *FileLink*.

When prompted, choose whether to embed the file into the *Plant Simulation* model or not:

- **Yes:** Copies the file into your *Plant Simulation* model file when you save the model file.

- **No:** Creates a link to the file in the file system of your computer.

Double-click the icon in the *Frame* to open the application assigned to the file type and the file for editing. If the link is invalid, *Plant Simulation* opens the dialog *FileLink* and shows the path and name of the invalid link in the text box **File Name**. You can open the dialog of the *FileLink* by right-clicking its icon and selecting **Open** on the context menu.

You can:

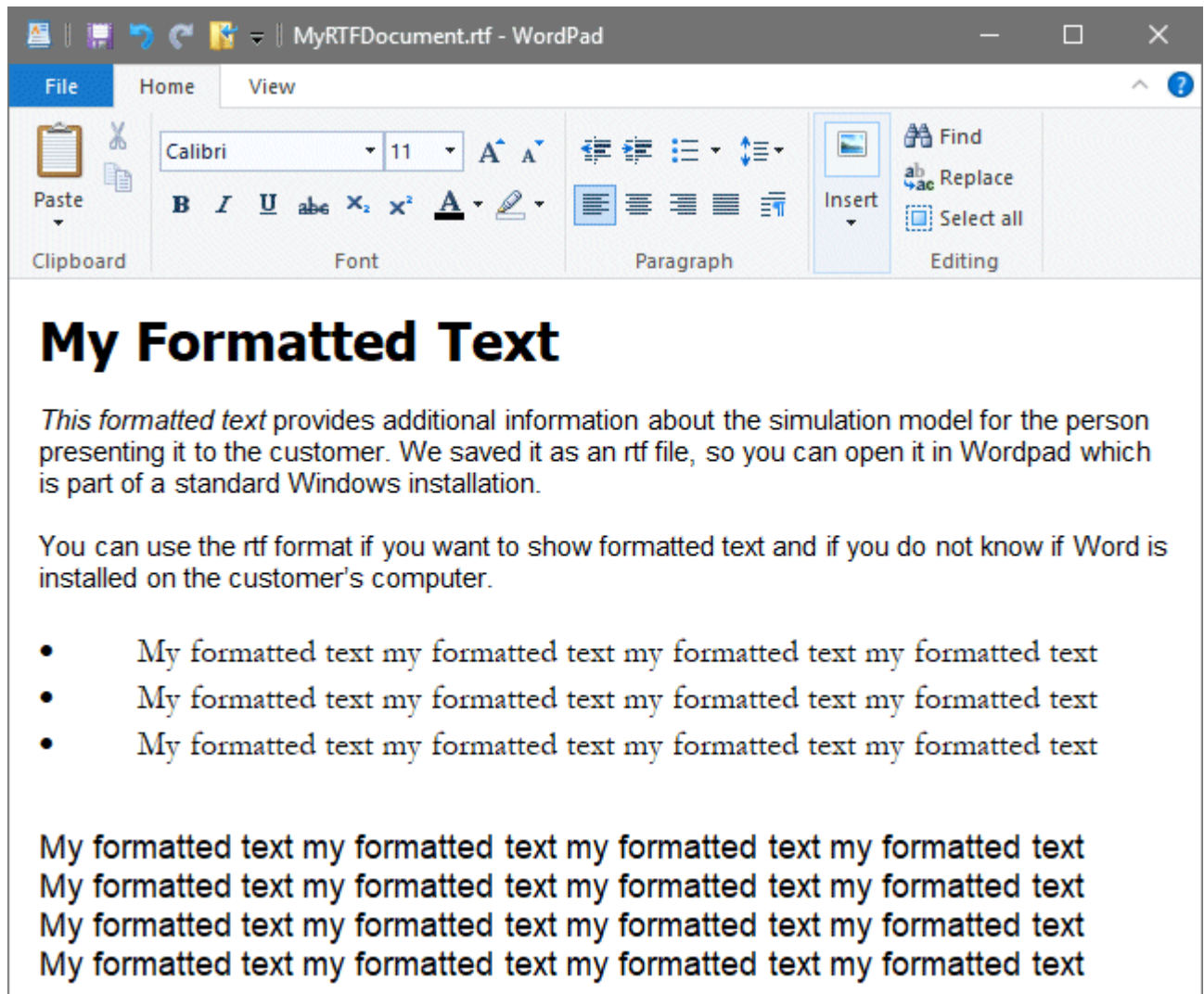
- [Open Formatted Text from within the Model](#)
- [Open a Picture From Within the Model](#)
- [Open Documents From Office Applications and PDF Files](#)
- [Play a Video in Your Simulation Model](#)

Open Formatted Text from within the Model

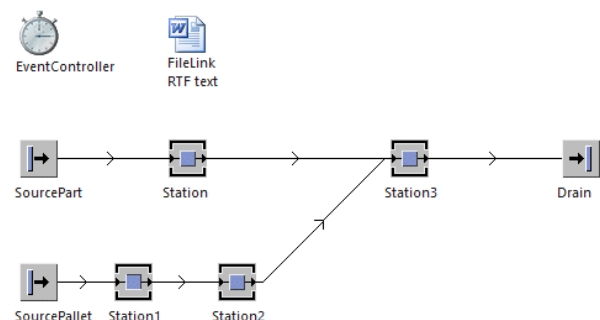
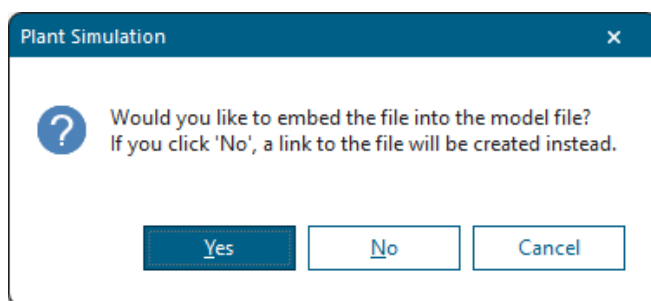
To show a document in your simulation model, which contains formatted text, you can save it as an RTF file. This way another person can always open this file in *WordPad*, which is part of a standard Windows installation.

To do so, proceed as follows:

- Create the document in a program which can save RTF files. We created our document in *WordPad*, entered our text, applied formatting, and saved it as an RTF file with the name *MyRTFDocument.rtf*.



- Drag the RTF file from the *Windows Explorer* to the *Frame*. If you want to embed the file into your simulation model, click **Yes**.



For demonstration purposes, we changed the **Label** so that it does not show the path of the file but RTF Text.

When you then pass the model on to another person, and that person double-clicks the icon of the object *FileLink*, it opens in *WordPad*.

Back to Display Data Stored in an External Program

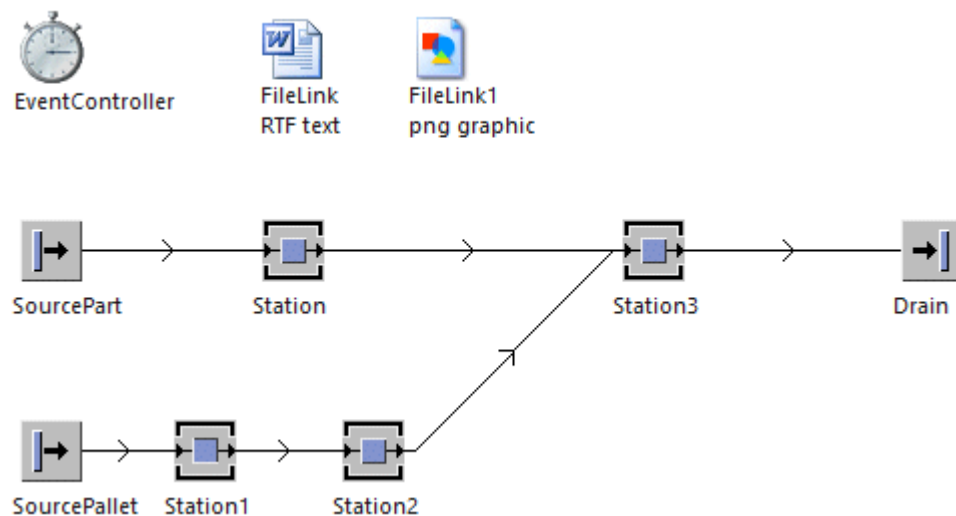
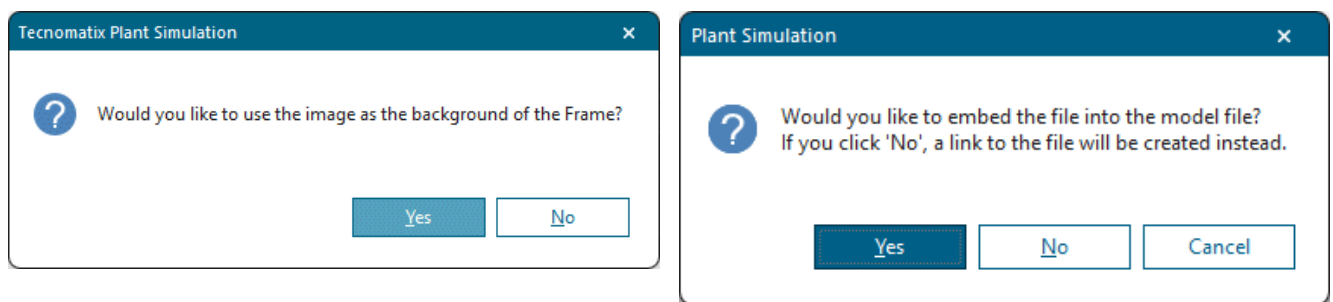
Open a Picture From Within the Model

To show a picture in your simulation model, you can save it as a BMP or as a PNG file. This way another person can always open this file in *Paint*, which is part of a standard Windows installation.

To do so, proceed as follows:

- Save the picture in a program which can save BMP or PNG files. We saved our picture in *Paint* and saved it as a PNG file with the name *MyPicture*.
- Drag the BMP or PNG file from the *Windows Explorer* to the *Frame*.

Plant Simulation asks you if you want to use the picture as the background image of your model. We clicked **No**. We then clicked **Yes** in the next dialog to embed the picture into our model.



For demonstration purposes, we changed the **Label** so that it does not show the path of the file but png graphic.

When you then pass the model on to another person, and that person double-clicks the icon of *FileLink1*, it opens the picture in *Paint*.

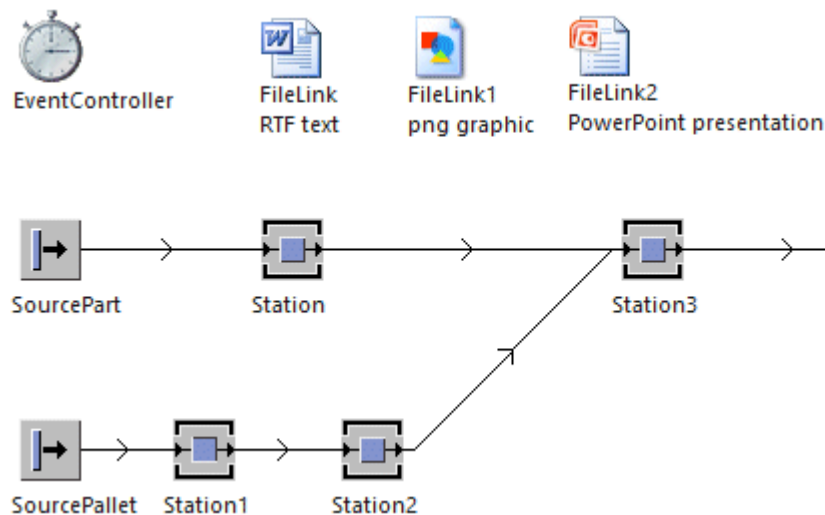
Back to **Display Data Stored in an External Program**

Open Documents From Office Applications and PDF Files

To show a document from Microsoft Office applications or PDF files created from them in your simulation model, save it in the native format or as a PDF file. Other people can only open these types of files, when Microsoft Office and a PDF Viewer are installed on their computer.

To do so, proceed as follows:

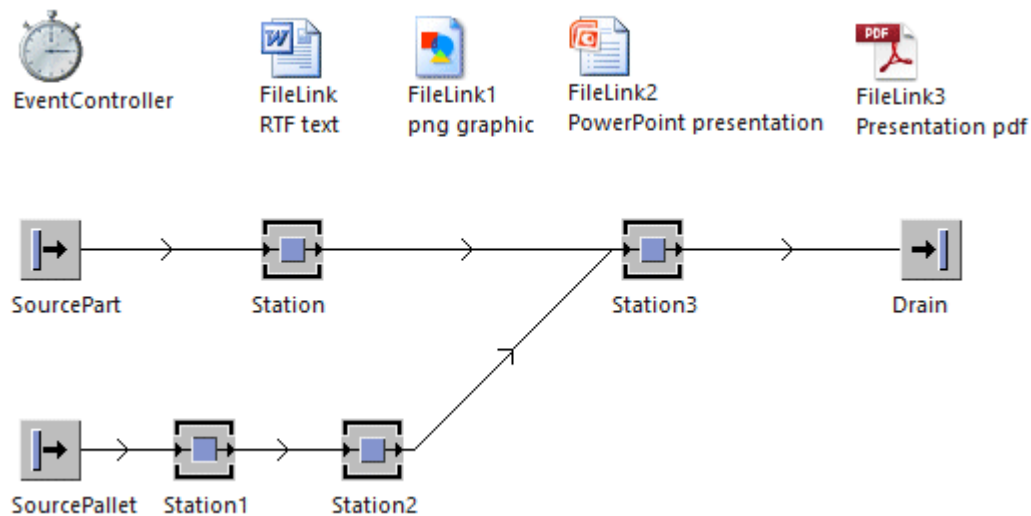
- Save the Microsoft Office document in the native format of the respective application. We saved our presentation with the name *MyPresentation* in *PowerPoint* first as a Microsoft Office PowerPoint Presentation. Then we also saved it as a PDF file.
- Drag the presentation, i.e., the PPTX file, from the *Windows Explorer* to the *Frame*. Click **Yes** in the dialog to embed the presentation into the model.



For demonstration purposes, we changed the **Label** so that it does not show the path of the file but *PowerPoint presentation*.

When you then pass the model on to another person, and that person double-clicks the icon of *FileLink2*, it opens the presentation in *PowerPoint*.

- Drag the PDF file created from the presentation file from the *Windows Explorer* to the *Frame*. Click **Yes** in the dialog to embed the PDF file of the presentation into the model.



For demonstration purposes, we changed the **Label** so that it does not show the path of the file but **Presentation pdf**.

When you then pass the model on to another person, and that person double-clicks the icon of *FileLink3*, it opens the presentation in the PDF Viewer that is installed on the computer.

Back to [Display Data Stored in an External Program](#)

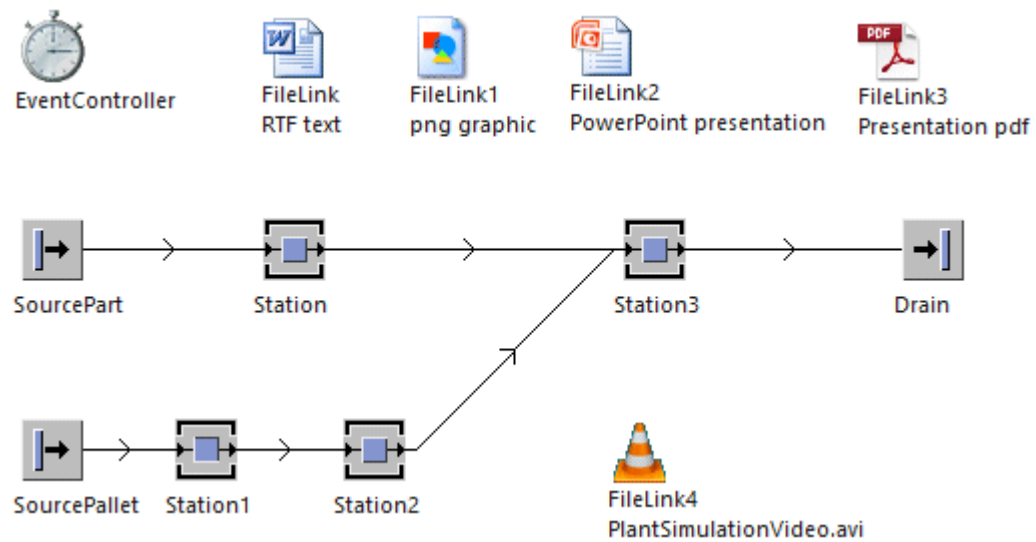
Play a Video in Your Simulation Model

To embed and play a video in a simulation model with the default media player, proceed as follows:

- Record the video in a video recording tool and save it as an .avi file.

We recorded a worker who places parts into a work cell and then removes the processed parts from it. We named our video file *MyVideo*.

- Drag the video, i.e., the AVI file, from the *Windows Explorer* to the *Frame*. Click **Yes** in the dialog to embed the video into the model.
- Our simulation model now looks like this.



To play the video, double-click the object *FileLink4*. The video then plays with the default player that is set on your computer.

